SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKIH TEHNOLOGIJA OSIJEK

Sveučilišni prijediplomski studij Računarstvo

PROGRAMSKO RJEŠENJE WEB SUSTAVA I USPOREDNA ANALIZA POSTUPAKA STVARANJA PREPORUKA FILMOVA ZASNOVANIH NA FILTRIRANJU I STROJNOM UČENJU

Završni rad

Anto Kožarević

Osijek, 2025.

Sadržaj

1 UVOD 1

1.1 Zadatak završnog rada 2

2 PREGLED STANJA U PODRUČJU SUSTAVA ZA PREPORUKE FILMSKIH SADRŽAJA 3

2.1 Razvoj i primjena sustava preporuka 3

2.2 Izazovi preporučivanja filmova 3

2.3 Pregled postojećih rješenja za filtriranje 4

2.3.1 Netflix 4

2.3.2 Youtube 5

2.3.3 IMDb 6

2.4 Korišteni pristupi i funkcionalnosti u rješenjima 8

3 MODEL I GRAĐA WEB SUSTAVA 9

3.1 Funkcionalni zahtjevi na web sustav 9

3.2 Nefunkcionalni zahtjevi na web sustav 9

3.3 Metode stvaranja preporuka 10

3.1.1 Filtriranje prema sadržaju 10

3.1.2 Kolaborativno filtriranje 11

3.3.3 Strojno učenje u preporučivanju filmova 14

3.4 Građa web sustava 15

4 RAZVOJ I PROGRAMSKA IMPLEMENTACIJA WEB SUSTAVA 16

4.1 Pregled tehnologija za razvoj aplikacije 16

4.2 Programske tehnologije korištene u razvoju sustava 16

4.2.1 Programski jezici i okviri 17

4.2.2 Baze podataka SQLite 18

4.2.3 Alati za klijentsku i poslužiteljsku stranu aplikacije 18

4.3 Odabir baze filmskih podataka 19

4.3.1 Opis baze podataka SQLite 19

4.3.2 Način povezivanja s aplikacijom 20

4.4 Programsko rješenje funkcionalnosti web sustava 21

4.4.1 Registracija i prijava korisnika 21

4.4.2 Pohrana korisničkih profila 22

4.4.3 Sučelje za unos preferenci i za prikaz preporuka 24

4.5 Integracija predloženog modela stvaranja preporuka 25

4.5.1 Obrada podataka 25

4.5.2 Treniranje modela 26

4.5.3 Generiranje preporuka 30

5 PRIKAZ RADA S ANALIZOM SUSTAVA I USPOREDBOM POSTUPAKA PREPORUČIVANJA 33

5.1 Prikaz korištenja web sustava 33

5.2 Usporedba filtriranja prema sadržaju, kolaborativnog filtriranja i modela strojnog učenja 34

5.3 Analiza kvalitete preporuka 36

5.4 Ograničenja implementacije 37

6 ZAKLJUČAK 39

7 LITERATURA 40

SAŽETAK 42

ABSTRACT 43

ŽIVOTOPIS 44

PRILOZI 45

1 UVOD

Filmska industrija danas nudi iznimno veliku količinu sadržaja, zbog čega korisnici digitalnih platformi često dolaze do problema odabira filmova koji najbolje odgovaraju njihovim interesima i sklonostima. Sustavi preporuka igraju ključnu ulogu u rješavanju problema jer olakšava korisnicima pronalazak relevantnih naslova, žanra čime podiže kvalitetu njihovog iskustva korištenja. Ovi sustavi se temelje na naprednim algoritma i metodama, uključujući umjetnu inteligenciju, strojno učenje i analizu podataka, kako bi stvorili personalizirane prijedloge sadržaja. Jedan od glavnih izazova predstavlja prezasićenost informacijama, gdje korisnici teško dolaze do sadržaja koji ih doista zanima. Popularnost algoritama za preporuke, kakvi se koriste na poznatim platformama za filmove, naglašava na važnost sustava koji prilagođavaju sadržaj svakom pojedincu. Danas se tehnologije za stvaranje preporuka primjenjuju u raznim područjima, od zabave do obrazovanja, a njihova primjena stalno raste.

Ovaj završni rad bavi se razvojem web aplikacije za preporuku filmova te objedinjuje tri glavna pristupa: filtriranje prema sadržaju, kolaborativno filtriranje i metode strojnog učenja. Cilj rada je istražiti i usporediti navedene pristupe u kontekstu kvalitete preporuke filmova, analizirati njihove prednosti i ograničenja te prikazati praktičnu implementaciju kroz web aplikaciju. Web aplikacija je izrađena korištenjem programskog jezika Python, web okvira *Flask* te tehnologija programskog rješenja na strani korisnika HTML, CSS, *JavaScript* i *Bootstrap*. Analiza rada aplikacije provedena je kroz funkcionalno testiranje i usporedbu rezultata različitih modela preporuka.

Poglavlje 2 predstavlja pregled stanja u području sustava za preporuke filmskih sadržaja, uključujući razvoj, izazove i postojeća rješenja, kao i korištene tehnologije. Poglavlje 3 prikazuje funkcionalne i nefunkcionalne zahtjeve sustava, metode stvaranja preporuka te građu i arhitekturu web aplikacije. Poglavlje 4 detaljno opisuje proces razvoja i programsku implementaciju sustava, korištene tehnologije, strukturu baze podataka te implementaciju funkcionalnosti i algoritama preporuka. Poglavlje 5 daje prikaz rada aplikacije uz analizu učinkovitosti pojedinih metoda preporuka, usporedbu rezultata i identifikaciju ograničenja implementacije.

Rad je strukturiran tako da najprije donosi pregled stanja i postojećih rješenja, potom opisuje zahtjeve i arhitekturu sustava, tehnološku implementaciju, analizu rada aplikacije te na kraju daje zaključke i prijedloge za daljnja unaprjeđenja.

1.1 Zadatak završnog rada

U završnom radu treba opisati i analizirati ciljeve i izazove preporučivanja filmskih sadržaja, te analizirati stanje u području i postojeća slična rješenja s naglaskom na različite mogućnosti stvaranja preporuka postupcima filtriranja i strojnog učenja. Nadalje, potrebno je izabrati bazu filmskih sadržaja i omogućiti njeno spajanje s web aplikacijom. Također, treba izabrati odgovarajući skup podataka za filtriranje, treniranje i vrednovanje modela strojnog učenja, kao i postupak stvaranja preporuka filtriranjem prema sadržaju i kolaborativnim filtriranjem, obaviti predobradbu podataka, treniranje i vrednovanje modela strojnog učenja. Navedene postupke treba programski implementirati u web aplikaciju koja treba omogućiti registriranje i prijavu korisnika, unos i pohranu profila korisnika i značajki za stvaranje preporuka na navedene načine, te prikaz preporuka filmskih sadržaja. Predložene postupke filtriranja i strojnog učenja treba analizirati i usporediti, a web aplikaciju ostvarenu u aktualnim programskim tehnologijama i jezicima ispitati i analizirati za odgovarajuće ulazne podatke i različite profile i sklonosti korisnika.

2 PREGLED STANJA U PODRUČJU SUSTAVA ZA PREPORUKE FILMSKIH SADRŽAJA

U ovom poglavlju daje se pregled razvoja, izazova i postojećih rješenja sustava za preporuku filmskih sadržaja. Opisane su temeljne metode i algoritmi koji se koriste u sustavima za preporuku, analiziraju se izazovi s kojima se sustavi suočavaju te se predstavljaju najpoznatiji primjeri iz prakse.

2.1 Razvoj i primjena sustava preporuka

Sustavi za preporuku filmova predstavljaju ključan alat za personalizaciju korisničkog iskustva, jer omogućuju korisnicima jednostavnije otkrivanje sadržaja koji je u skladu s njihovim interesima. Prvi oblici ovih sustava su se pojavili početkom 1990-ih, prvenstveno za filtriranje e-pošte i informacija. Danas su sustavi za preporuku postali neizostavni element digitalnih platformi, od online trgovina do *streaming* usluga. Razvoj umjetne inteligencije i strojnog učenja značajno je unaprijedio kvalitetu i preciznost preporuka. Uz odabir odgovarajućih algoritama, uspjeh tih sustava važan je i način prikupljanja podataka, razumijevanje korisničkih navika i aplikacijskog okruženja. Primjena personaliziranih preporuka nalazi se u brojnim platformama ako što su *Amazon*, *Pandore* i *Netflix*, pri čemu se značajan dio prihoda ostvaruje upravo zahvaljujući sustavima za preporuke. Osim zabavne industrije, sustavi za preporuku se sve više nalaze i u drugim područjima, primjerice u obrazovanju, gdje pomažu u prilagodbi sadržaja prema potrebama svakog učenika.

2.2 Izazovi preporučivanja filmova

Postoje različiti izazovi s kojima se suočavaju sustavi za preporuke. Među najčešćima su problem *hladnog pokretanja* (engl. *cold start*), nedostatak podataka (engl. *data sparsity*) i skalabilnost (engl. *scalability*). Kod hladnog pokretanja, sustavu je potreban dovoljan broj korisnika kako bi mogao pronaći podudaranja. Primjerice, ako želimo identificirati korisnika koji ima slične interese ili sličan predmet, uspoređujemo ih s postojećim skupom korisnika ili predmeta. U početnoj fazi, novi korisnik nema popunjen profil jer još nije ocijenio nijedan predmet, pa sustav ne zna njegov ukus, što otežava davanje preporuka. Sličan problem se javlja i kod novog predmeta, jer ga nijedan korisnik nije ocijenio, budući da je nov za sve. Ovi problemi mogu se riješiti primjenom hibridnih tehnika [1]. Kod problema rascjepkanost podataka radi se o tome da matrica korisnika ili ocjena je vrlo rijetka. Teško je pronaći korisnike koji su ocijenili iste predmete, jer većina korisnika ne

ocjenjuje sadržaj. Stoga je teško pronaći skup korisnika koji su ocijenili određene predmete. Davanje preporuka postaje iznimno teško kada sustav raspolaže s malo informacija o korisniku. Skalabilnost predstavlja dodatni problem jer kolaborativno filtriranje koristi ogromnu količinu podataka kako bi bilo pouzdanije i bolje, što zahtijeva više računalnih resursa. Kako količina informacija eksponencijalno raste, obrada postaje skupa i dolazi do netočnih rezultata zbog izazova velikih podataka (engl. *big data*) [2]. Ostatak ovog rada organiziran je na sljedeći način: U potpoglavlju 1 raspravljaju se vrste tehnika filtriranja i njihovi podtipovi, uz detaljan pregled i usporedbu. Također se obrađuju mjere sličnosti i metričko vrednovanje. Nadalje, opis skupa podataka nalazi se u potpoglavlju 3. Na kraju, članak završava raspravom o prednostima, izazovima i ograničenjima rada te se u potpoglavlju 4 daje pogled na buduće mogućnosti.

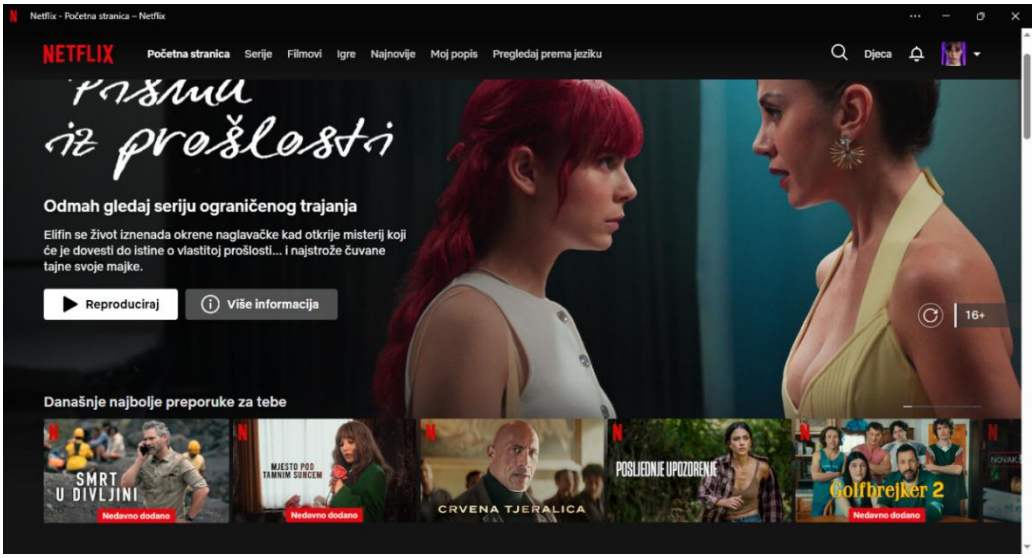
2.3 Pregled postojećih rješenja za filtriranje

Sustavi preporuke filmskih sadržaja danas su dio gotovo svih većih platformi za pregled video sadržaja. Njihov cilj je pomoći korisnicima u otkrivanju relevantnih filmova i serija na temelju sklonostima, povijesti gledanja i ponašanja sličnih korisnika. Postojeća rješenja primjenjuju različite metode filtriranja, najčešće temeljene na kolaborativnom i sadržajnom pristupu, ali i sve više pažnje uzimaju napredne metode strojnog učenja, duboke neuronske mreže i hibridne pristupe. U nastavku analiziraju se tri najznačajnija sustava: *Netflix* [3], *YouTube* i *IMDb*. Svaka od navedenih platformi primjenjuje vlastite algoritme i pristupe kako bi korisnicima pružila što relevantnije preporuke, pri čemu se oslanjaju na velike količine korisničkih podataka i sofisticirane modele predikcije. U nastavku se prikazuje usporedni pregled njihovih funkcionalnosti i korištenih metoda filtriranja, s naglaskom na tehnološke aspekte i razlike u implementaciji sustava preporuka.

2.3.1 Netflix

Prema [4], *Netflix* je globalna streaming platforma temeljena na pretplatnom modelu koja omogućuje gledanje filmova i serija putem internetski povezanih uređaja. Kada govorimo o sustava preporuka, *Netflix* se smatra jedno od najpoznatijih i najrazvijenijih rješenja, jer se aktivno i intenzivno oslanja na napredne algoritme kako bi personalizirao iskustvo svakog korisnika. Preporuke se generiraju analizom korisničkog ponašanja, poput povijesti gledanja, učestalost i vrijeme gledanja, vrste sadržaja koje korisnik preferira, kao i ponašanja korisnika sa sličnim interesima. Primijenjeni pristupi obuhvaćaju kolaborativno filtriranje, filtriranje prema sadržaju te hibridne metode koje u posljednje vrijeme uključuju i tehnike strojnog učenja te duboke neuronske mreže. *Netflix* ne samo da predlaže sadržaj, već i prilagođava naslovnice, redoslijed prikaza filmova i serija unutar sučelja. Prema službenim podacima *Netflixa*, više od 80 % sadržaja koji

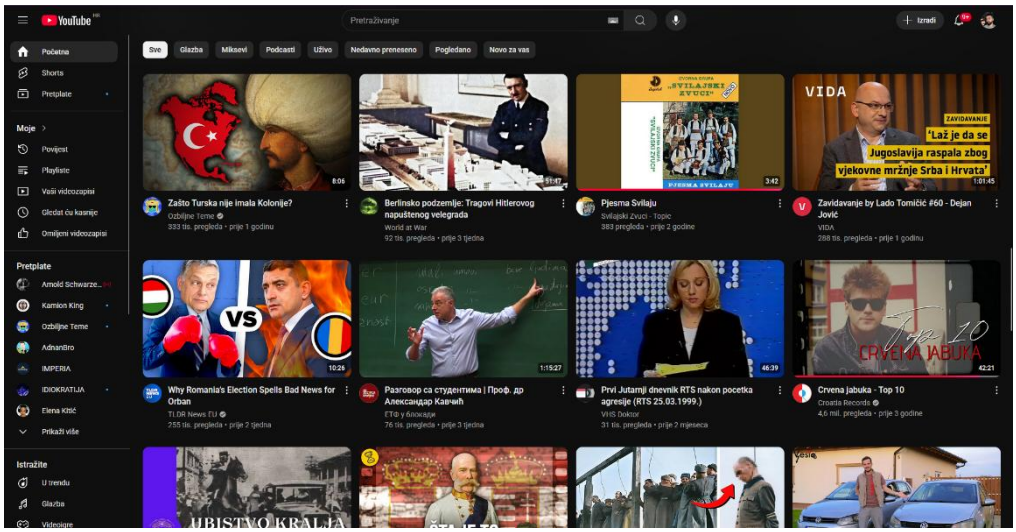
korisnici gledaju dolazi upravo putem preporuka generiranih ovim sustavom. Pregled *Netflix* stranice prikazan je na slikama 2.2.



Slika 2.3 Početna stranica Netlfix-a.

2.3.2 Youtube

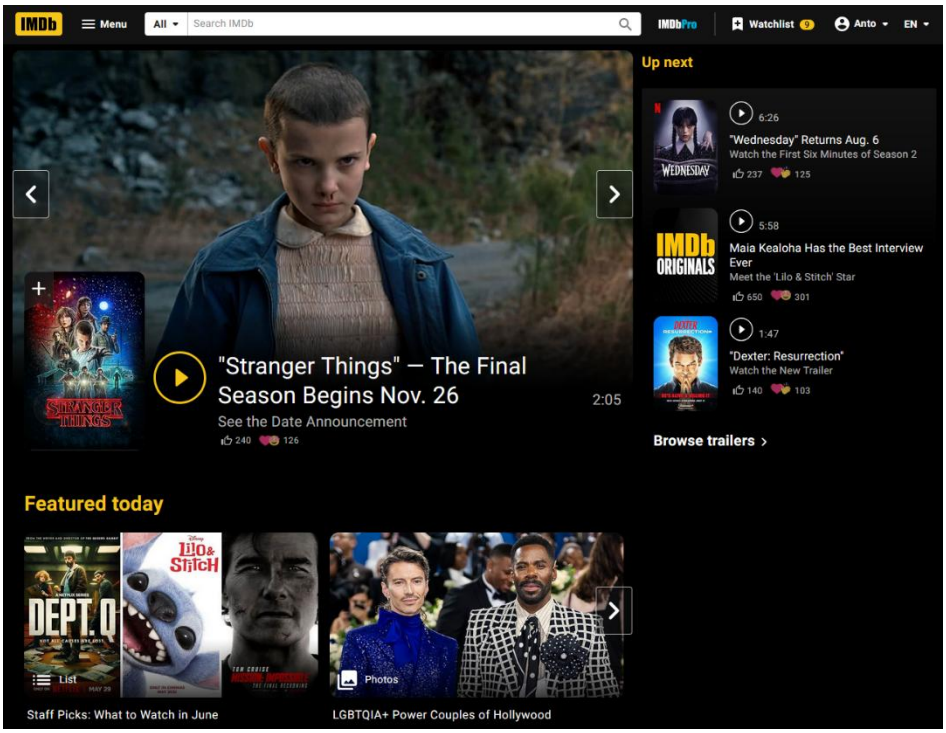
YouTube [5] je jedna od najpopularnijih platformi za dijeljenje videozapisa, koja omogućuje korisnicima pretraživanje, gledanje, komentiranje i prijenos vlastitih sadržaja. Iako primarno služi kao otvoreni sustav za distribuciju video sadržaja, *YouTube* također posjeduje razvijen mehanizam za preporuke koji je ključan za korisničko iskustvo. Sustav preporuka na *YouTubeu* koristi kombinaciju sadržajnog i kolaborativnog filtriranja, pri čemu algoritmi analiziraju različite signale korisničkog ponašanja. To uključuje povijest gledanja, trajanje gledanja pojedinog videa, broj klikova, lajkove, pretplate i interakcije s drugim korisnicima. Osim toga, *YouTube* koristi metode strojnog učenja za modeliranje korisničkih interesa i personalizaciju sadržaja prikazanih na početnoj stranici, kao i u odjeljcima *Preporučeno za vas* i *Sljedeće za gledanje*. Poseban izazov u kontekstu filtriranja predstavlja količina sadržaja, na platformu se svakodnevno prenose stotine sati videa u minuti, što otežava preporuku relevantnih, a posebno sigurnih i prikladnih sadržaja. Kao odgovor na te izazove, *YouTube* je razvio dodatne mehanizme poput sigurnog načina rada (engl. *Safety Mode*), roditeljskih kontrola te posebno izdvojene aplikacije *YouTube Kids*, koja koristi strože algoritme filtriranja i sadržaje prikladne za djecu. Unatoč kompleksnosti i količini sadržaja, *YouTubeov* sustav preporuka ostaje jedan od najutjecajnijih u smislu usmjeravanja korisničke pažnje i poticanja angažmana, a njegova sposobnost personalizacije uvelike se oslanja na tehnike dubokog učenja i neprekinuto treniranje modela na velikim količinama podataka. Pregled *Youtube* stranice prikazan je na slici 2.4.



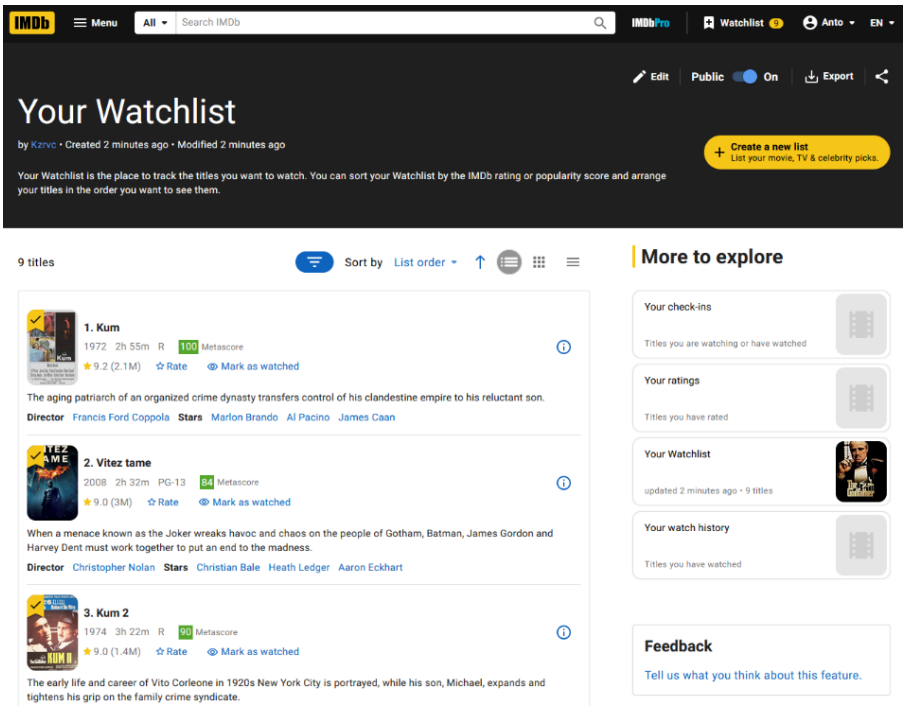
2.4 Početna stranica YouTube

2.3.3 IMDb

Internet Movie Database (*IMDb*) [6] je jedna od najpoznatijih i najopsežnijih internetskih baza podataka posvećenih filmskim i televizijskim sadržajima. Osnovana 1990. godine, *IMDb* pruža informacije o milijunima filmova, serija, glumaca, redatelja i drugih profesionalaca iz industrije zabave. Korisnici mogu pretraživati sadržaje, ocjenjivati ih, pisati recenzije te stvarati vlastite liste za gledanje. U kontekstu sustava preporuka, *IMDb* koristi kombinaciju korisničkih ocjena i recenzija kako bi generirao preporuke za gledatelje. Sustav ocjenjivanja temelji se na skali od 1 do 10, a rezultati se prikazuju kao ponderirani prosjeci, što omogućuje preciznije rangiranje sadržaja prema popularnosti i kvaliteti. Ove ocjene često služe kao referentna točka za korisnike pri odabiru filmova ili serija za gledanje. Osim toga, *IMDb* nudi i dodatne usluge poput *IMDbPro*, namijenjene profesionalcima u industriji, koja pruža detaljnije informacije o projektima, kontaktima i tržišnim trendovima. Ova platforma igra značajnu ulogu u povezivanju korisnika s relevantnim sadržajem te u oblikovanju percepcije o popularnosti i kvaliteti filmskih i televizijskih djela. Pregled *IMDb* stranice prikazan je na slikama 2.6. i 2.7.



Slika 2.6 Početna stranica IMDb



Slika 2.7 Prikaz liste gledanja filmova

2.4 Korišteni pristupi i funkcionalnosti u rješenjima

Sustavi za preporuku [7] u video i filmskim platformama za streaming temelje se na različitim tehnikama filtriranja podataka i algoritamskim pristupima. Ključne funkcionalnosti ovih sustava uključuju automatsko preporučivanje sadržaja, personalizaciju korisničkog sučelja, analizu korisničkog ponašanja te poticanje angažmana i zadržavanja korisnika.

Najčešće korišteni pristupi u preporukama uključuju [8]:

- Filtriranje prema sadržaju (engl. *content-based filtering*): preporuke se temelje na sličnostima među karakteristikama sadržaja (npr. žanr, redatelj, glumci, ključne riječi) koje je korisnik prethodno gledao i ocjenjivao.

- Kolaborativno filtriranje (engl. *collaborative filtering*): sustav analizira preferencije i ponašanja većeg broja korisnika te na temelju međusobnih sličnosti preporučuje sadržaj (korisnici slični vama su također gledali...).

- Hibridni pristupi: kombiniraju više metoda (npr. sadržajno i kolaborativno filtriranje) radi postizanja veće točnosti i robusnosti preporuka.

Konkretno, *Netflix* koristi hibridni sustav za preporuke koji se temelji na višeslojnim neuronskim mrežama za obradu korisničkih podataka i izradu personaliziranih prijedloga. *YouTube* [9] primjenjuje duboko učenje i algoritme rangiranja kako bi preporučio videozapisa, pri čemu u obzir uzima karakteristike kao što su gledanost, trajanje sesije i obrasci ponašanja gledatelja. *IMDb* se u većoj mjeri oslanja na sustav recenzija i ocjenjivanja, ali koristi i jednostavnije mehanizme za preporuku sličnih filmova i izradu korisničkih listi. Ovi sustavi se kontinuirano poboljšavaju primjenom strojnog učenja, čime se prilagođavaju promjenama u sklonostima i navikama korisnika te povećavaju relevantnost preporučenog sadržaja.

3 MODEL I GRAĐA WEB SUSTAVA

Sustavi preporuka su razvijeni s ciljem da korisnicima olakšaju pregled svih dostupnih podataka i pronađu sadržaj koji im odgovara. U ovom slučaju aplikacije video i filmske platforme provode proces usklađivanja u kojem spajaju serije i filmove. Temelj svake preporuke čini metoda na kojoj se sustav temelji, pri čemu metode mogu varirati u principu rada, rezultatima i složenosti. Ovdje će se raspravljati o tri ključne skupine metoda za generiranje preporuka; filtriranje temeljeno na sadržaju, kolaborativno filtriranje i pristupi temeljeni na strojnom učenju. Često kombinacija više od jednog pristupa daje bolje rezultate i stoga se koristi nekoliko različitih hibridnih modela.

3.1 Funkcionalni zahtjevi na web sustav

Web sustav za preporuku filmova omogućuje korisnicima registraciju i prijavu, pregled i uređivanje profila, ocjenjivanje filmova, pretraživanje i pregled informacija o filmovima, dodavanje filmova na popis omiljenih te prikaz osobnog popisa gledanih filmova. Sustav stvara personalizirane preporuke pomoću korisničkih ocjena, a korisnicima su dostupni i najnoviji te najbolje ocijenjeni filmovi iz baze. Pažnja je posvećena sigurnosti podataka i jednostavnosti korištenja, uz modularnu strukturu koja omogućuje proširenje funkcionalnosti.

3.2 Nefunkcionalni zahtjevi na web sustav

Nefunkcionalni zahtjevi [10] obuhvaćaju opće značajke web aplikacije koje osiguravaju njenu učinkovitost, sigurnost i fleksibilnost. Sustav mora osigurati brzo vrijeme odziva kako bi korisničko iskustvo bilo ugodno, s generiranjem preporuka unutar nekoliko sekundi čak i pri većem broju korisnika i ocjena. Zaštita podataka, jedan je od ključnih zahtjeva. Lozinke se čuvaju u kriptiranom obliku, a pregled profila i personaliziranim sadržaja dostupan je isključivo prijavljenim korisnicima. Podaci korisnika nisu dostupni trećim stranama i sustav je zaštićen od najčešćih sigurnosnih prijetnji poput ubrizgavanja strukturiranog jezika upita (engl. structured query language - SQL i napada putem skripti (engl. cross-site scripting - XSS). Sučelje je dizajnirano tako da je lako za korištenje, omogućujući korisnicima brzo snalaženje i interakciju sa sustavom. Aplikacija je organizirana modularno, što omogućuje jednostavno održavanje i daljnje nadogradnje. To uključuje mogućnost dodavanja novih algoritama za preporuke, povezivanje s dodatnim bazama podataka te integraciju vanjskih usluga. Sustav je izrađen korištenjem modernih web tehnologija koje jamče stabilnost, skalabilnost i dugoročnu održivost rješenja.

3.3 Metode stvaranja preporuka

U ovom dijelu rada prikazane su glavne metode koje se koriste za generiranje preporuka filmova u sustavu. Svaka metoda ima specifičan pristup analizi korisničkih podataka i karakteristika filmova, a u nastavku se detaljno opisuju: kolaborativno filtriranje, filtriranje prema sadržaju i algoritmi strojnog učenja.

3.1.1 Filtriranje prema sadržaju

Filtriranje prema sadržaju [8] (engl. *content-based filtering*) jedan je od osnovnih pristupa u sustavima preporuka, a temelji se na analizama osobina sadržaja koje je korisnik ranije ocijenio pozitivno ili negativno. Pretpostavka ovog pristupa jest da će korisnici preferirati stavke koje su slične onima koje su im se sviđale ranije. Glavni cilj je prepoznati obrasce u korisničkim interesima i te informacije koristiti za preporuku novog sadržaja koji je sličan onome što je korisnik već ocijenio ili gledao. Za razliku od kolaborativnog filtriranja, koje se oslanja na podatke o drugim korisnicima, filtriranje sadržaja fokusira se isključivo na individualne interese i ponašanje korisnika. Zbog toga sustav ne mora imati podatke o drugim korisnicima kako bi generirao neke preporuke, što je osobito korisno u slučajevima kada je baza korisnika nova ili ograničena.

Filtriranje sadržaja radi na vrlo jednostavnoj ideji, a to je da će se korisniku svidjeti sadržaj koji ima slične karakteristike sadržaju koji je prethodno ocijenio ili na drugi način konzumirao. Sustav kreira korisnički profil iz analize sadržaja koji je korisnik ocijenio ili pogledao ili s njim na neki drugi način komunicirao. Ovaj profil može uključivati informacije kao što su:

- najgledaniji žanrovi (drama, akcija, komedija),
- stalni glumci, redatelji ili teme,
- ključne riječi iz opisa serija i filmova,
- ocjene koje je korisnik dao nekim sadržajima.

Sadržaj je predstavljen atributima filma, koji mogu biti strukturirani (npr. metapodaci poput žanra, redatelja ili godine proizvodnje filma) ili nestrukturirani (npr. tekstualni opisi). Sustav potom koristi tehnike kao što su modeli vektorskog prostora (npr. TF-IDF) i mjere sličnosti (npr. kosinusna sličnost) kako bi pronalazio novi sadržaj koji odgovara atributima koji odgovaraju korisnicima. Primjerice, ako korisnik često gleda psihološke trilere i akcijske filmove s određenim

glumcem, sustav će mu preporučiti drugačije filmove koji su sličnog žanra s istim glumcem koje korisnik još nije ni gledao.

Prednosti filtriranja prema sadržaju [11] su:

- Personalizacija: Sustav direktno prilagođava interesima pojedinog korisnika, dajući preporuke koje bolje odgovaraju sklonostima i navikama korisnika.
- Nezavisnost od zajednice: Nije potreban velik broj korisnika niti njihove interakcije i podaci, što omogućuje rad i u ranoj fazi razvoja sustava.
- Relevantnost: Interesi pojedinih korisnika [12] eksplicitno su definirani sustavom na kojem se temelje preporuke. To obično dovodi do visoke preciznosti u tome koje filmove i serije prikazati.

Nedostaci filtriranja prema sadržaju [13] su:

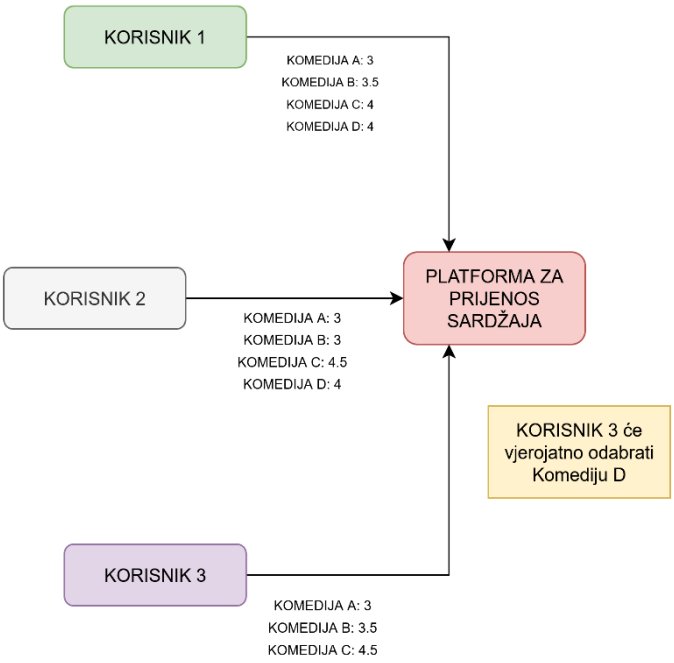
- Nedostatak raznolikosti: Sustav može zapasti u takozvanom *filter bubble* gdje se korisniku stalno prikazuje previše sličan sadržaj onome što korisnik već voli i ništa novo, što smanjuje šanse pronalaska različitog i novog sadržaja.
- Ograničena pokrivenost: Ako sadržaj nema dovoljno detaljne deskriptivne značajke i opise (npr. slab opis metapodataka), sustav će imati male šanse za ispravno modeliranje sličnosti i stoga pružanje relevantnih preporuka za filmove i serije.
- Ignoriranje popularnosti i društvenih trendova: Sustav ne koristi informacije o visoko ocijenjenom i popularnom sadržaju među drugim korisnicima sličnih interesa. To može značiti kako korisnik propušta trenutno popularne prijedloge.

3.1.2 Kolaborativno filtriranje

Kolaborativno filtriranje [14] temelji se na ideji kako će korisnici kojima su se svidjele iste stavke u prošlosti vjerojatno imati slične interese i u budućnosti. Kolaborativno filtriranje ne analizira sadržaj stavki, nego se oslanja na korisničke interakcije poput ocjena ili pregleda kako bi pronašlo obrasce ponašanja i generiralo preporuke. Sustavi koji koriste ovaj pristup ne trebaju informacije o samom sadržaju, već se oslanjaju isključivo na ponašanje korisnika i time omogućuju preporučivanje i potpuno različitih stavki koje povezuje interes korisnika. Ipak se ovaj pristup suočava s izazovima poput problema hladnog starta i rijetkosti podataka.

Filtriranje temeljeno na korisnicima [15] (engl. *user-based collaborative filtering*) koristi pristup koji pretpostavlja kako će korisnici sa sličnim sklonostima u prošlosti i ubuduće pokazivati slične

interese. Metoda funkcionira usporedbom povijesnih podataka o ponašanju korisnika kako bi se identificirali oni čiji su obrasci ocjenjivanja sličniji. Sustav najprije prepoznaje skupinu korisnika čiji su ukusi slični s ciljanom osobom, a zatim preporučuje sadržaj koji su ti slični korisnici pozitivno ocijenili, a koji ciljani korisnik još nije konzumirao. Primjerice, ako su korisnik A i korisnik B ocijenili velik broj istih filmova na sličan način, sustav može korisniku A preporučiti film koji se korisniku B izrazito svidio, ali koji A još nije pogledao. Ova metoda omogućuje još detaljniju personalizaciju preporuka budući da izravno uzima u obzir stvarne interese korisnika, a ne samo karakteristike sadržaja. Iako je metoda relativno jednostavna za primjenu i često daje solidnu točnost kada postoji dovoljna razmjena između korisnika. No, kao i većina sustava preporuka, ovaj se pristup suočava s određenim izazovima. Glavni problemi uključuju ono što je poznato kao problem hladnog pokretanja; to je kada postoje ili novi korisnici ili nove stavke i nedovoljno podataka za generiranje vrijedne preporuke, a također i problem skalabilnosti, posebno u velikim sustavima koji imaju mnogo korisnika kao i sadržaja. Kao što je prikazano na slici 3.2 ako su dva korisnika pogledali isti film, a postoji i treći korisnik sličnog ukusa, vrlo vjerojatno da će mu se prikazati film koji su pogledali prva dva ili više korisnika ovisno o broju registriranih.



Slika 3.2 Primjer kolaborativnog filtriranja

Metoda kolaborativnog filtriranja temeljenog na stavkama [16] oslanja se na analizu odnosa između pojedinih filmova na temelju korisničkih ocjena. Osnovna pretpostavka je kako su filmovi

koje korisnici često ocjenjuju na sličan način međusobno slični, pa se korisniku mogu predložiti filmovi nalik onima koje je već favorizirao.

Proces preporuke uključuje sljedeće korake:

- Konstrukcija matrice korisnik–stavka: Prikupljaju se sve korisničke ocjene i organiziraju u matricu u kojoj redci predstavljaju korisnike, a stupci filmove.
- Izračun sličnosti između stavki: Parovi filmova uspoređuju se pomoću neke od gore navedenih metrika ili kosinusne sličnosti, *Pearsonove* korelacije ili poboljšane metode poput prilagođene kosinusne sličnosti koja uzima u obzir prosječne ocjene korisnika.
- Predviđanje ocjene: Algoritam zatim predviđa ocjene koje je korisnik dao onim filmovima koje nije ocijenio tako što prvo pronalazi skup sličnih filmova koje je korisnik ocijenio, a zatim pomoću tih ocjena predviđa ocjenu za svaki ciljni film. Može se temeljiti na ponderiranoj sredini ili regresijskom modelu ali sve to ovisi o količini podataka.

Prednosti kolaborativnog filtriranja su:

- Učinkovitost: Budući da je ukupni broj filmova manji od broja korisnika, matrica sličnosti među stavkama zauzima manje prostora i može se unaprijed izračunati, što znatno ubrzava proces generiranja preporuka.
- Skalabilnost: Metoda lako prilagođava rad s velikim skupovima podataka jer se izračuni odnose ma međusobne sličnosti stavki, a ne korisnika.
- Visoka kvaliteta preporuka: U praksi daje točnije prijedloge preporuka od modela temeljenih na korisnicima, osobito u slučajevima kada su korisnički profili rijetki ili promjenjivi.

Ograničenja kolaborativnog filtriranja:

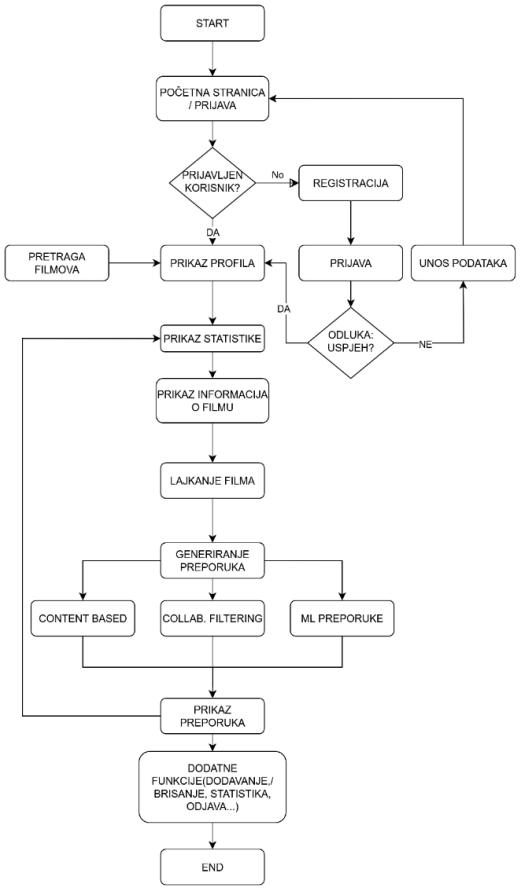
- Problem s hladnim startom: Novi filmovi bez ocjena teško ulaze u preporuke jer sustav nema podataka o njima koji bi mjerili njihovu sličnost.
- Ograničeni kontekst: Oslanja se samo na povijest ocjena korisnika bez korištenja podataka poput žanra, redatelja ili opisa sadržaja.
- Potreba za redovitim ažuriranjem: Dodavanje velikog broja novih ocjena zahtijeva ponovno izračunavanje matrice sličnosti, što može biti značajno opterećenje za sustav.

3.3.3 Strojno učenje u preporučivanju filmova

Uloga strojnog učenja u sustavima za preporuku filmova [17] danas je od presudne važnosti u modernim web aplikacijama, ponajprije zbog rastućih količina dostupnih podataka i potrebe za personalizacijom. Korištenjem naprednih algoritmima strojnog učenja, moguće je modelirati složene obrasce korisničkog ponašanja, prepoznati skrivene veze između korisnika i sadržaja te generirati preporuke s većom preciznošću. Među najčešće primijenjenim metodama su latentni faktorski modeli, kao što su singularna dekompozicija vrijednosti (engl. *Singular Value Decomposition* - SVD) [18] i vjerojatnosno faktoriziranje matrice, koji korisničko-filmskih interakcije razlažu na apstraktne dimenzije interesa. Uz njih, sve su popularniji i modeli temeljeni na neuronskim mrežama, posebno auto-enkoderima [19], sve se češće koriste za nelinearno mapiranje korisničkih sklonosti. U sustavima gdje je cilj predvidjeti ocjene filmova, korisni su i regresijski pristupi te *Bayesovi* modeli koji unose dodatnu fleksibilnost kroz probabilističko razmišljanje. Ključna faza u primjeni strojnog učenja je normalizacija podataka. Prije treniranja modela, podaci o ocjenama često se standardiziraju uklanjanjem prosječne ocjene svakog korisnika ili filma kako bi se ublažila osobna pristranost. U slučajevima u kojima su podaci implicitni (na primjer, sadržaj koji se vidi bez eksplicitne ocjene), podaci se često pretvaraju u binarne radi lakšeg rukovanja. Učinkovitost modela ocjenjuje se prema mnogim pokazateljima. Mjerni podaci kao što su srednja apsolutna pogreška (engl. *Mean Absolute Error* - MAE) i korijen srednje kvadratne pogreške (engl. *Root Mean Squared Error* - RMSE) koriste se za mjerenje točnosti predviđanja. Procjena preporuka kao popisa obično se provodi pomoću metrika kao što su preciznost pri K (engl. *Precision@K*), dohvat pri K (engl. *Recall@K*) i normalizirani diskontirani kumulativni dobitak (engl. *normalized discounted cumulative Gain* – NDCG). U procjeni učinkovitosti ovih sustava koriste se metrike koje mjere koliko su preporučeni filmovi relevantni stvarnim sklonostima korisnika, te u kojem se redoslijedu preporučuju. Primjenom ovih pristupa, sustavi postaju ne samo točniji već i robusniji na šumove i nepotpune podatke, čime se korisnicima osigurava personalizirano i kvalitetno iskustvo konzumacije sadržaja [20].

3.4 Građa web sustava

Na slici 3.1 prikazan je slijed korištenja funkcionalnosti koje su opisane u prethodnim potpoglavljima.



Slika 3.3 Prikaz slijeda korištenja funkcionalnosti web aplikacije za preporučivanje filmova

Kao što je prikazano na slici 3.3, korisničko iskustvo započinje na početnoj stranici gdje korisnik odabire želi li se registrirati ili prijaviti. Proces registracije i prijave je povezan s validacijom podataka, dobiva jasnu povratnu informaciju i mogućnost ponovnog unosa. Nakon uspješne autentifikacije, pristupa se korisničkom profilu koji služi kao središnje mjesto interakcije unutar aplikacije. Korisnički profil omogućuje pregled osobne statistike te pristup funkciji pretraživanja filmova. Preporuke se prikazuju kroz tri algoritma – prema sadržaju, prema ponašanju sličnih korisnika i putem strojnog učenja – čime se korisniku omogućuje raznolika i relevantna ponuda filmova. Dodatne funkcionalnosti obuhvaćaju administrativne opcije poput dodavanja i brisanja filmova, pregled aktivnosti te odjavu.

4 RAZVOJ I PROGRAMSKA IMPLEMENTACIJA WEB SUSTAVA

Razvoj i implementacija web sustava za preporuku filmova je izgrađen dijelovima planiranja arhitekture, odabir odgovarajućih tehnologija i izradu funkcionalnosti koje korisnicima pružaju jednostavno i intuitivno iskustvo. Ovo poglavlje prikazuje cjelokupan proces izrade web sustava, odabir programskih jezika, baze podataka do implementacije funkcionalnosti i sustava za preporuke. Opisano je kako su pojedini dijelovi sustava povezani i koji su tehnički izazovi pratili razvoj. Naglasak je stavljen na sigurnost, modularnost i jednostavnost korištenja, s ciljem stvaranja aplikacije za buduća proširenja i održavanje aplikacije.

4.1 Pregled tehnologija za razvoj aplikacije

Programsko rješenje na strani poslužitelja razvijeno je u programskom jeziku Python, pri čemu je za izradu i upravljanje web aplikacijom korišten web okvir *Flask*. On omogućuje jednostavno definiranje API ruta za komunikaciju s klijentskim dijelom aplikacije. Podaci su pohranjeni u *SQLite*, koja omogućuje brzu integraciju i nudi jednostavno upravljanje informacijama o korisnicima, filmovima i njihovim ocjenama. Klijentski dio aplikacije razvijen je korištenjem HTML-a za strukturiranje sadržaja, dok su *CSS* i *Bootstrap* primijenjeni za oblikovanje vizualnog dizajna sučelja. *JavaScript* je iskorišten za dinamičku interakciju s korisnikom te za komunikaciju sa poslužiteljem putem *AJAX* zahtjeva. Ova kombinacija tehnologija omogućila je razvoj prilagodljivog sučelja prilagođenog različitim vrstama ekrana. Što se tiče implementacije i obrade algoritama preporuke filmova korištene su dodatne Python biblioteke, uključujući one za autentifikaciju korisnika, obradu podataka te primjenu naprednih modela strojnog učenja. Poseban naglasak je stavljen na modularnu strukturu sustava jednostavnost nadogradnje funkcionalnosti i zaštita podataka. Odabrane tehnologije omogućile su učinkovitu fazu razvoja, testiranja i implementacija komponenti aplikacije koji su ključni za rad, osiguravajući pouzdanost i kvalitetu korisničkog doživljaja.

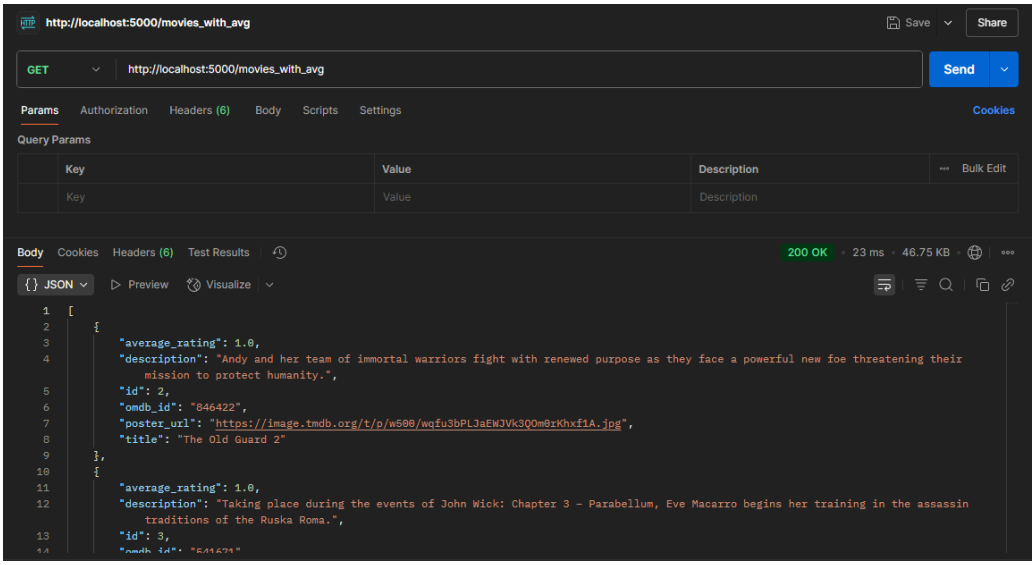
4.2 Programske tehnologije korištene u razvoju sustava

U ovom poglavlju opisane su glavne programske tehnologije koje su korištene pri izradi web sustava za preporuku filmova. Prikazani su odabrani programski jezici, web okviri, baza podataka te alati koji su korišteni na klijentskoj i poslužiteljskoj strani aplikacije. Prikazom ovih tehnologija predstavlja se uvid u arhitekturu i funkcionalnost sustava.

4.2.1 Programski jezici i okviri

Poslužiteljski dio aplikacije razvijen je u programskom jeziku Python [21], cijenjen po jednostavne i pregledne sintakse, velik broj biblioteka za web razvoj i analizu podataka. Kao glavni web okvir odabran je *Flask* [22], koji pojednostavljuje izradu i upravljanje web aplikacijama, omogućuje izradu ruta te povezivanje s bazom podataka putem sustava preslikavanja objekata na relacijsku bazu podataka (engl. *Object-Relational Mapping* – ORM). Klijentska strana aplikacije izrađena je korištenjem HTML-a [9] za strukturu stranica, CSS-a [9] za vizualno oblikovanje korisničkog sučelja te *JavaScripta* za dinamičku interakciju s korisnikom i komunikaciju s poslužiteljem putem *AJAX* [23] poziva. Programski jezici i okviri su odabrani zbog svoje široke podrške u zajednici, dostupne dokumentacije i jednostavne integracije.

Slika 4.1 prikazuje primjer korištenja *Postman* alata za slanje *GET* zahtjeva prema API ruti */movies_with_avg*. Ova ruta vraća podatke o filmovima iz baze podataka zajedno s izračunatom prosječnom ocjenom svakog filma. U tijelu odgovora prikazuju se informacije kao što su naslov filma, opis, identifikacijski brojevi, *URL* postera te prosječna ocjena, a rezultat je prikazan u JSON formatu. Ovakav prikaz omogućuje jednostavno testiranje ispravnosti API-ja i pregled vraćenih podataka prije prikaza korisnicima na web sučelju a slika 4.2 prikazuje izvadak iz programskog koda poslužiteljskog dijela aplikacije, gdje je definirana *Flask* ruta */movies_with_avg*. Funkcija *movies_with_avg()* obrađuje GET zahtjeve na toj ruti i vraća JSON podatke o svim filmovima, uključujući prosječne ocjene, pozivajući pomoćnu funkciju *get_all_movies_with_avg()*. Ovaj kod ilustrira način na koji se API implementira u aplikaciji i kako se podaci iz baze dohvaćaju i šalju prema korisničkom dijelu sustava.



Slika 4.1 Rezultat GET zahtjeva na /movies_with_avg – dohvat filmova iz baze preko Postmana.

```python
@app.route('/movies_with_avg', methods=['GET'])
def movies_with_avg():
    return jsonify(get_all_movies_with_avg())
```

Slika 4.2 Primjer API rute za dohvat filmova iz baze (GET /movies_with_avg).

4.2.2 Baze podataka SQLite

Za pohranu podataka u ovoj web aplikaciji koristi se relacijska baza podataka *SQLite*, poznata po jednostavnoj integraciji s Pythonom i brzom implementacijom za manje i srednje velike projekte. *SQLite* pohranjuje sve podatke unutar jedne datoteke, što dosta olakšava lokalni razvoj i testiranje funkcionalnosti aplikacije. Struktura baze prilagođena je zahtjevima sustava koja sadrži tablice za korisnike, filmove, ocjene i rezultate preporuka. Detaljniji opis strukture i organizacije baze, uključujući nazive tablica i njihove veze, dan je u potpoglavlju 4.2.1.

4.2.3 Alati za klijentsku i poslužiteljsku stranu aplikacije

Poslužiteljska strana aplikacije izrađena je u programskom jeziku Python, zahvaljujući njegovoj jednostavnoj sintaksi, bogatoj kolekciji paketa i širokoj podršci za obradu podataka te razvoj web aplikacija. Za razvoj web okvira korišten je *Flask*, koji omogućava jednostavno kreiranje API ruta, povezivanje s bazom podataka te modularan razvoj i razvoj funkcionalnosti sustava. Za autentifikaciju korisnika, obradu podataka, rad s bazom i implementaciju algoritama za preporuku, korišteni su razni Python paketi, *flask-login*, *pandas*, *sqlite3* i biblioteka *surprise*, namijenjena modelima strojnog učenja.

Podaci se pohranjuju u *SQLite*, koja omogućava lokalno spremanje informacija o korisnicima, filmova, ocjena i rezultata preporuka, a integracija s Python programskim jezikom je jednostavna i efikasna.

Klijentska strana aplikacije izrađena je pomoću HTML-a za strukturiranje sadržaja, CSS-a za vizualno oblikovanje te programskog okvira *Bootstrap,* radi izrade prilagodljiv i vizualno privlačnog korisničkog sučelja prilagođenog svim uređajima. Za dinamičku interakciju i komunikaciju s poslužiteljskim dijelom korišten je *JavaScript*, uz *AJAX* pozive za dohvat i prikaz podataka iz baze. Također su korištene su i dodatne *JavaScript* biblioteke za poboljšanje interaktivnosti web sustava.

4.3 Odabir baze filmskih podataka

Za potrebe rada web aplikacije za preporuku filmova, bilo je nužno odabrati prikladnu bazu podataka koja omogućuje jednostavno spremanje, dohvat i upravljanje filmskim podacima, korisnicima i njihovim ocjenama. Kriteriji odabira uključivali su jednostavnost integracije, podršku za relacijske podatke, mogućnost proširenja te stabilan rad u manjim i srednje velikim aplikacijama.

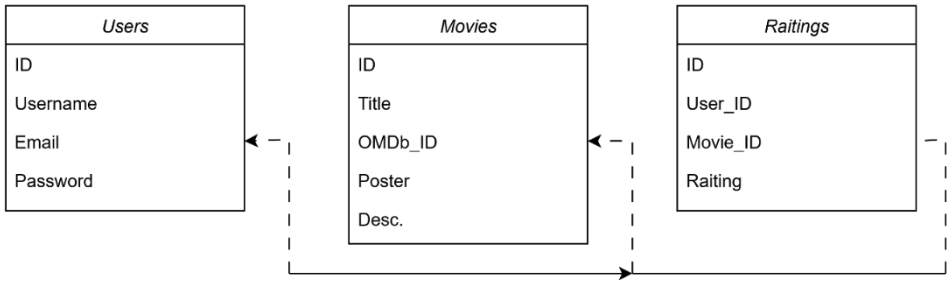
4.3.1 Opis baze podataka SQLite

Za omogućavanje rada s bazom podataka odabran je ***SQLite***. *SQLite* je popularna relacijska baza podataka koja se razlikuje od većine drugih baza jer se svi podaci pohranjuju u jednu datoteku na disku, bez potrebe za zasebnim poslužiteljem. Ova karakteristika čini je iznimno pogodnom za razvoj i testiranje manjih web aplikacija, kao i za prototipove.

Struktura baze uključuje tri osnovne tablice:

- *users* — tablica u kojoj se pohranjuju podaci o korisnicima (korisničko ime, e-mail, *hashirana* lozinka)
- *movies* — tablica s podacima o filmovima (naslov, OMDB ID, URL postera, opis filma)
- *ratings* — tablica koja povezuje korisnike i filmove te pohranjuje korisničke ocjene za pojedini film.

Kako je i prikazano na slici 4.3 tablice su povezane putem stranih ključeva, čime se omogućuje praćenje koji je korisnik ocijenio koji film i kojom ocjenom. Ovakva relacijska struktura omogućuje jednostavno filtriranje, pretraživanje i analizu podataka.



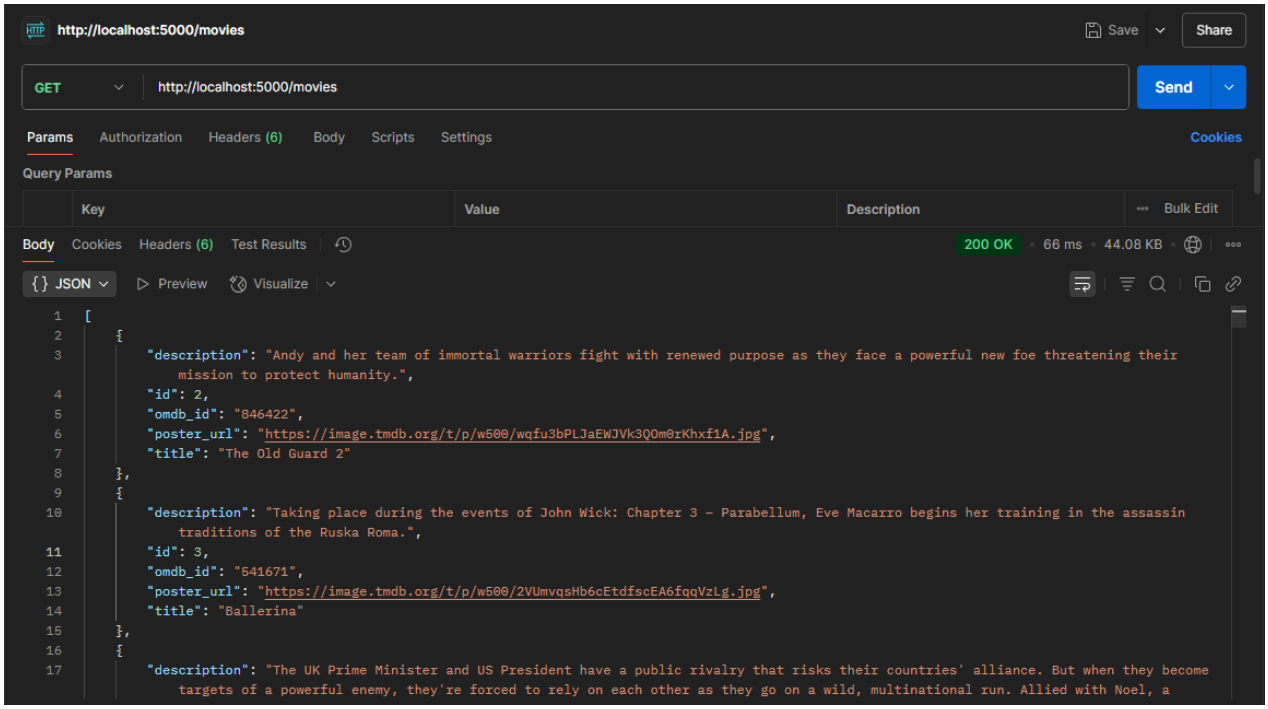
Slika 4.3 ER dijagram baze podataka aplikacije s prikazom tablica i njihovih veza.

Na dijagramu su prikazane i veze (relacije) između tablica. Svaki korisnik može ocijeniti više filmova, a svaki film može imati više ocjena, čime se ostvaruje veza više-prema-više između

korisnika i filmova preko posredne tablice *ratings*. Omogućuje se brz uvid u organizaciju podataka, povezanost tablica te način na koji se implementira funkcionalnost aplikacije poput registracije korisnika, dodavanja filmova i ocjenjivanja.

4.3.2 Način povezivanja s aplikacijom

Aplikacija pristupa bazi podataka korištenjem standardne Python biblioteke *sqlite3*, što omogućuje izravno izvršavanje SQL upita i manipulaciju podacima. Sve operacije nad bazom (dodavanje korisnika, dohvat filmova, spremanje ocjena, itd.) implementirane su kroz posebne funkcije u odvojenom modulu (npr. database.py). Za komunikaciju između klijenta i poslužitelja koristi se *REST API* kako je i prikazano na slici 4.4 i 4.5 i implementiran u okviru *Flask* web okvira. programsko rješenje na strani poslužitelja definira API rute poput /*movies* (za dohvat svih filmova), /rate_*movie* (za ocjenjivanje filma) i dr. Kada korisnik putem sučelja pošalje zahtjev (npr. klikom na gumb ili unosom podataka), klijentska aplikacija šalje HTTP zahtjev poslužitelju, koji obrađuje podatke, komunicira s bazom i vraća odgovor u JSON formatu. Ovakav način povezivanja omogućuje jasnu podjelu između prezentacijskog i podatkovnog sloja, sigurnu razmjenu podataka i jednostavno proširenje funkcionalnosti.



Slika 4.4 Prikaz Postman zahtjeva na krajnju pristupnu točku /movies koji prikazuje dohvat svih filmova iz baze.

```python
@app.route('/movies', methods=['GET'])
def movies():
    return jsonify(get_all_movies())
```

Slika 4.5 Primjer API rute za dohvat svih filmova iz baze (GET /movies).

4.4 Programsko rješenje funkcionalnosti web sustava

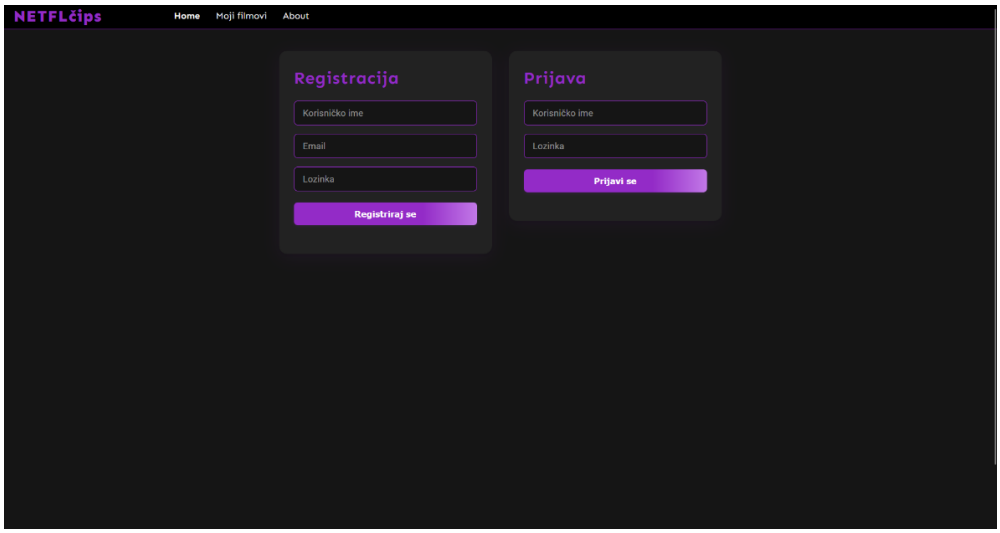
U ovom pod poglavlju opisuje se implementacija glavnih funkcionalnosti web aplikacije za preporuku filmova. Programsko rješenje obuhvaća autentikaciju korisnika, upravljanje korisničkim profilima, te sučelje za unos preferenci i prikaz preporuka.

4.4.1 Registracija i prijava korisnika

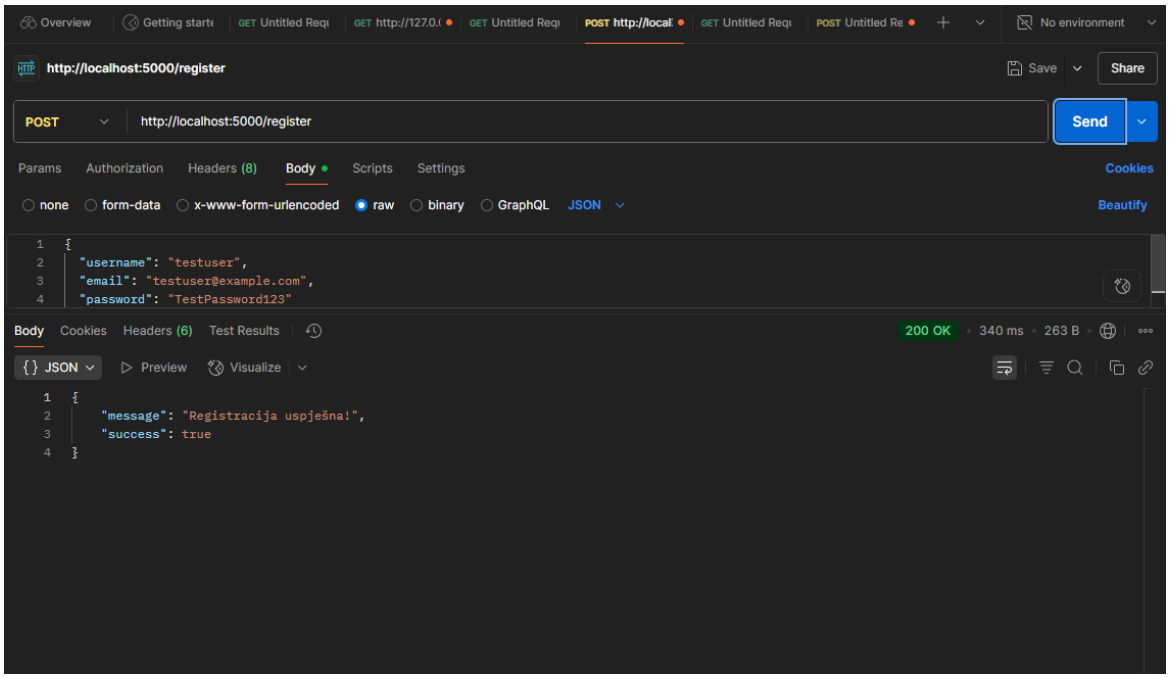
Registracija i prijava predstavljaju temeljne funkcionalnosti svake web aplikacije s korisničkim računima. Korisnici imaju mogućnost izrade vlastitog računa putem registracijskog obrasca, u kojem se unose osnovni podaci: korisničko ime, email i lozinka. Prilikom registracije, aplikacija provodi validaciju podataka, provjerava jedinstvenost email adrese te *hashira* lozinku radi zaštite korisničkih podataka. Kada se korisnik prijavljuje, unosi email i lozinku u prijavni obrazac. Programsko rješenje na strani poslužitelja aplikacije koristi funkcije za dohvat korisnika iz baze te uspoređuje *hashirane* lozinke kako je i prikazano u slici 4.6. Uspješnom prijavom korisniku se dodjeljuje sesija ili JSON Web Token (JWT), koji se koristi za autentikaciju kod daljnjih zahtjeva prema API-ju. Na slici 4.7 je prikazan zaslon za prijavljivanje u web aplikaciji a na 4.8 API zahtjev za registraciju.

```python
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    email = data.get('email')
    password = data.get('password')
    if not username or not email or not password:
        return jsonify({'success': False, 'message': 'Sva polja su obavezna.'}), 400
    success = add_user(username, email, password)
    if success:
        return jsonify({'success': True, 'message': 'Registracija uspješna!'})
    else:
        return jsonify({'success': False, 'message': 'Korisničko ime ili email već postoji.'}), 409
```

Slika 4.6 Primjer API rute za registraciju korisnika.



Slika 4.7 Prikaz registracijskog i prijavnog obrasca u web sučelju.



Slika 4.8 Primjer testiranja registracije novog korisnika korištenjem Postman alata. PUTEM POST zahtjeva na rutu /register.

4.4.2 Pohrana korisničkih profila

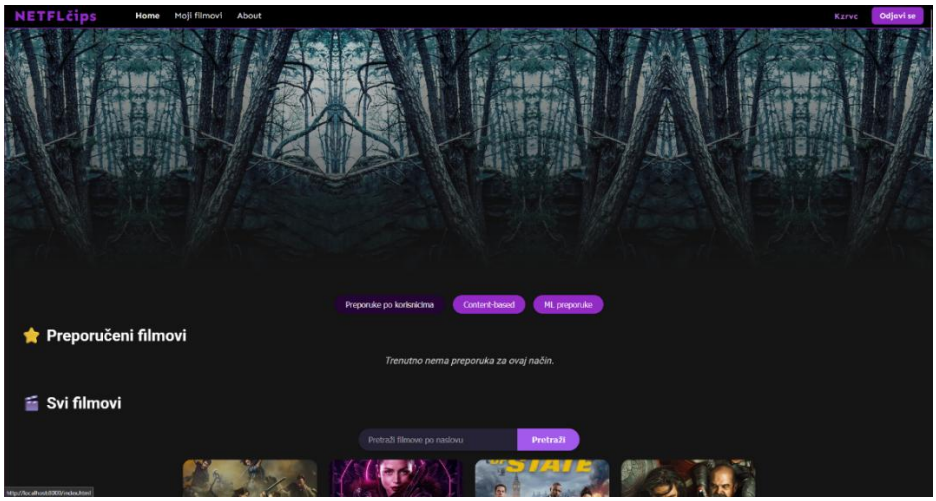
Podaci o korisnicima pohranjuju se u tablicu baze podataka *users*. Svaki korisnik ima jedinstveni identifikator, korisničko ime, email adresu i *hashiranu* lozinku kako je i prikazano na slici 4.9. Osim osnovnih podataka, profil može sadržavati informacije o prethodnim ocjenama filmova i sklonostima korisnika. Korisnički profili omogućuju personalizaciju aplikacije, npr. prikaz prethodnih ocjena, preporuka temeljenih na osobnim sklonostima, te kasnije uređivanje podataka. Kada korisnik zatraži svoj profil, programsko rješenje na strani poslužitelja dohvaća podatke iz

baze i šalje ih u JSON formatu klijentu, gdje se prikazuju u sučelju aplikacije. Na slici 4.10 je prikazan početni zaslon prijavljenog korisnika, a na slici 4.11 je prikazan dohvat korisnika pomoću API zahtjeva.

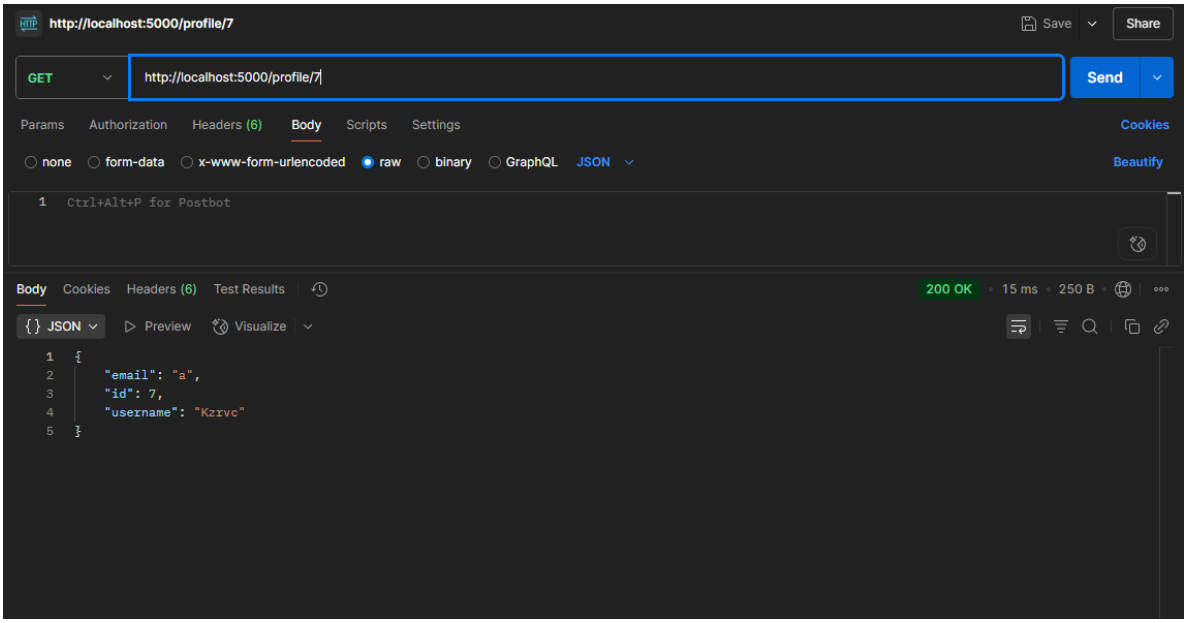
```python
@app.route('/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    user = get_user_by_id(user_id)
    if user:
        return jsonify(user), 200
    else:
        return jsonify({"error": "Korisnik nije pronađen"}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

Slika 4.9 Primjer API rute za dohvat korisnika.



Slika 4.10 Prikaz korisničkog profila na web sučelju.



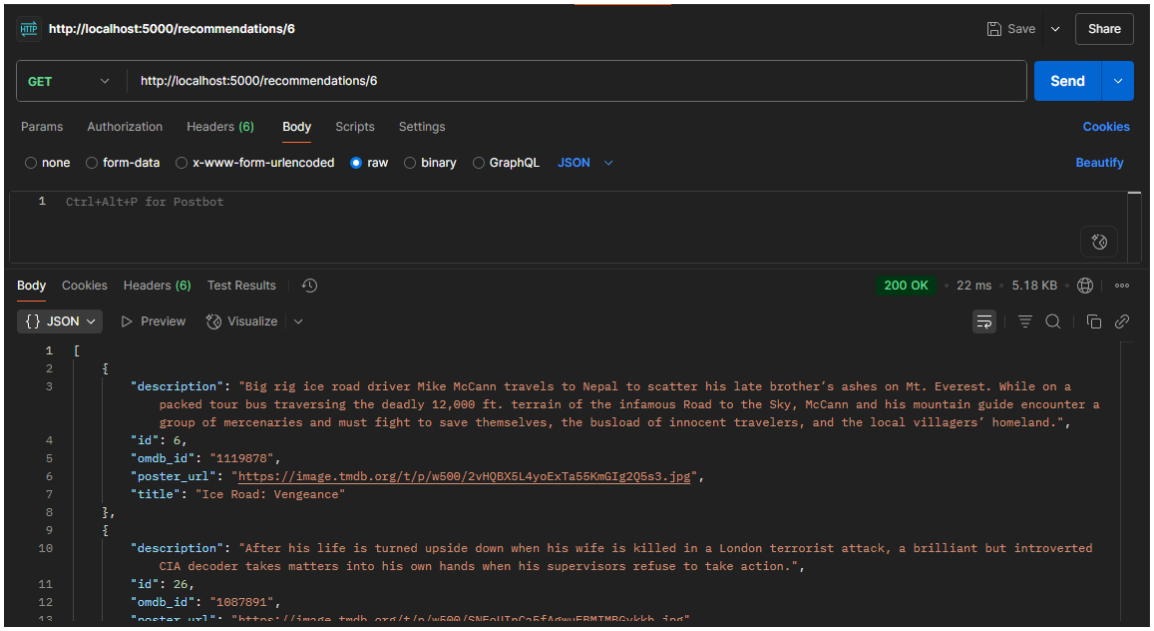
Slika 4.11 Prikaz Postman zahtjeva za /profile/7

4.4.3 Sučelje za unos preferenci i za prikaz preporuka

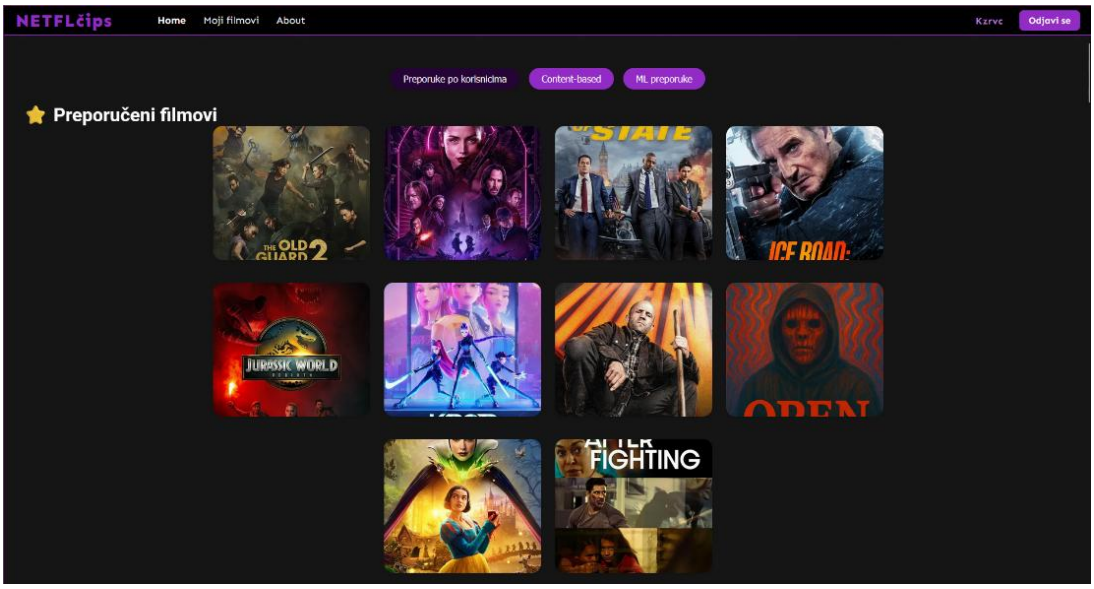
Sučelje za unos preferenci omogućuje korisniku odabrati svoje omiljene žanrove, glumce ili ocijeni nekoliko filmova, čime se poboljšava kvaliteta preporuka. Unesene preference i ocjene spremaju se u bazu podataka, najčešće u tablice *ratings* i/ili posebne tablice za preference. Na temelju ovih podataka, aplikacija pokreće algoritam za preporuku filmova. Algoritam uspoređuje korisničke preference s dostupnim filmovima i generira listu preporučenih naslova a na slici 4.12 i 4.13 je prikazan kod i API dohvat preporučenih filmova za pojedinog korisnika, također na slici 4.14 prikazan je zaslon web sučelja generiranih preporučenih filmova. Preporuke se korisniku prikazuju putem web sučelja, obično u obliku liste filmova sa slikama, nazivima i kratkim opisima.

```python
@app.route('/recommendations/<int:user_id>', methods=['GET'])
def recommendations_by_id(user_id):
    # Dohvati username na temelju user_id
    user = get_user_by_id(user_id)
    if not user:
        return jsonify({'success': False, 'message': 'Korisnik nije pronađen.'}), 404
    username = user['username']
    films = get_recommendations_for_user(username)
    return jsonify(films)
```

Slika 4.12 Dohvat preporuka za user_id.



Slika 4.13 Prikaz API odgovora sa listom preporuka.



Slika 4.14 Zaslon prikaza preporučenih filmova.

4.5 Integracija predloženog modela stvaranja preporuka

Integracija sustava preporuka u aplikaciju uključuje nekoliko koraka: prikupljanje i obradu podataka, treniranje odabranih modela te generiranje preporuka korisnicima u stvarnom vremenu. U nastavku su detaljno opisani svi koraci i način na koji su implementirani u aplikaciji.

4.5.1 Obrada podataka

Prvi korak u izgradnji sustava preporuka je kvalitetna obrada podataka. Podaci o korisnicima, filmovima i ocjenama pohranjeni su u *SQLite* bazi podataka. Podaci o filmovima preuzeti su s OMDB i TMDB baza, dok su ocjene generirane tijekom testiranja aplikacije i simulacijom korisničkih aktivnosti. Na slici 4.15 prikazano je dohvaćanje filmova pomoću *DB Browsera*, a na slici 4.16 je prikazan isječak koda za dohvaćanje podataka za model. Prilikom pokretanja funkcionalnosti preporuka, aplikacija iz baze dohvaća sve relevantne zapise:

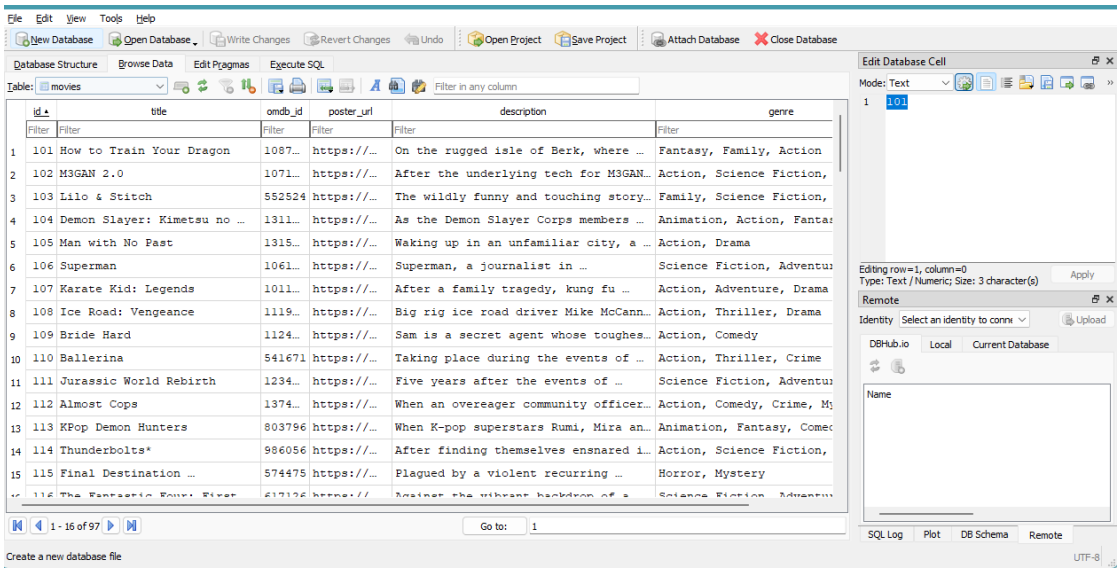
- Korisnici: svaka jedinstvena prijava registrira korisnika s korisničkim imenom i e-mail adresom.
- Filmovi: svaki film ima naslov, opis, poster i jedinstveni identifikator (TMDB/OMDB ID).
- Ocjene: veza između korisnika i filmova, gdje svaki korisnik može dati ocjenu od 1 do 5 za svaki film. Ove ocjene predstavljaju temelj za sve algoritme preporuka.

Podaci se zatim pripremaju za daljnju obradu, što uključuje:

- spajanje tablica radi dobivanja matrice korisnik–film,

- popunjavanje nedostajućih vrijednosti (npr. ako korisnik nije ocijenio film, tu vrijednost ostavljamo praznom ili postavljamo na 0),
- eventualnu normalizaciju ili transformaciju ocjena.

U slučaju preporuka na temelju sadržaja, koriste se opisi filmova koji se vektoriziraju (npr. TF-IDF metodom) kako bi se mogao izračunati stupanj sličnosti između filmova.



Slika 4.15 Prikaz tablice movies u DB Browseru s podacima o filmovima koji se koriste u aplikaciji, uključujući naslove, OMDB ID, URL postera, opise i žanrove.

```python
def get_all_movies():
    conn = sqlite3.connect('filmovi.db')
    c = conn.cursor()
    c.execute('SELECT id, title, omdb_id, poster_url, description FROM movies')
    movies = [
        {
            'id': row[0],
            'title': row[1],
            'omdb_id': row[2],
            'poster_url': row[3],
            'description': row[4]
        }
        for row in c.fetchall()
    ]
    conn.close()
    return movies
```

Slika 4.16 Isječak koda iz dijela programskog rješenja na strani poslužitelja za dohvaćanje podataka za model.

4.5.2 Treniranje modela

Nakon što su podaci pripremljeni, pristupa se procesu treniranja modela za preporuku filmova. U ovoj aplikaciji implementirana su tri različita pristupa kako bi se korisnicima omogućile što personaliziranije i relevantnije preporuke. Prvi pristup temelji se na kolaborativnom filtriranju,

gdje se analiziraju obrasci ponašanja korisnika kroz njihove ocjene filmova. Cilj je pronaći korisnike sa sličnim ukusima te na temelju njihovih pozitivnih ocjena predložiti filmove koje trenutni korisnik još nije pogledao. Ova metoda omogućuje otkrivanje novih naslova na temelju sklonostima korisničke zajednice, a u aplikaciji je realizirana korištenjem SQL upita i prilagođene logike na poslužiteljskoj strani. Na slici 4.17 se prikazuje dio poslužiteljski dio Flaska koji omogućuje dohvaćanje preporučenih filmova za korisnika putem rute */recommendations/<username>*. Funkcija *recommendations* dohvaća korisničko ime iz URL-a, poziva funkciju za generiranje preporuka na temelju tog korisnika, te rezultat vraća u JSON formatu koji korisnička strana sustava može prikazati. Na ovaj način povezujemo korisničke podatke te logiku preporuka s klijentskim dijelom aplikacije.

```python
@app.route('/recommendations/<username>', methods=['GET'])
def recommendations(username):
    films = get_recommendations_for_user(username)
    return jsonify(films)
```

Slika 4.17 Prikaz krajnje pristupne točke Flask koji vraća preporučene filmove za korisnika pozivanjem funkcije za generiranje preporuka.

Drugi pristup odnosi se na preporuke temeljene na sadržaju filmova, gdje se analiziraju opisi filmova. Svaki opis filma vektorizira se koristeći metodu učestalosti pojma – obrnuta učestalost u dokumentima (TF-IDF), što omogućuje usporedbu sličnosti između različitih filmova na temelju njihovih sadržajnih karakteristika. Kada korisnik pozitivno ocijeni određeni film, sustav mu preporučuje filmove čiji su opisi najsličniji prethodno ocijenjenim naslovima. Na slici 4.18 se prikazuje dio koda za filtriranje prema sadržaju. Prvo se dohvaćaju svi filmovi i ID-evi filmova koje je korisnik ocijenio. Zatim se iz opisa filmova generiraju tekstualni vektori pomoću TF-IDF metode. Izračunava se prosječni vektor ocijenjenih filmova kao korisnički profil, a zatim se izračuna sličnost tog profila s ostalim filmovima u bazi. Filmovi koje korisnik nije gledao rangiraju se prema sličnosti, a najrelevantniji se predlažu kao preporuka. Ovaj algoritam omogućuje da preporuke budu usmjerene na interese i afinitete pojedinačnog korisnika, neovisno o ponašanju drugih korisnika.

```python
@app.route('/content_recommendations/<username>', methods=['GET'])
def content_recommendations(username):
    all_movies = get_all_movies_with_avg()  # Vrati sve filmove s prosječnom ocjenom
    if not all_movies:
        return jsonify([])

    # Dohvati ID-eve filmova koje je korisnik ocijenio
    user_rated_ids = get_user_rated_movies(username)
    if not user_rated_ids:
        # Ako korisnik nema ocjena, vrati najpopularnije filmove (najveći prosjek)
        sorted_by_rating = sorted(all_movies, key=lambda x: x['average_rating'] or 0, reverse=True)
        return jsonify(sorted_by_rating[:12])

    # Tekst za sličnost: opis (i možeš dodati žanr ako ga imaš)
    def movie_text(movie):
        return movie.get('description', '') or ''

    movie_ids = [movie['id'] for movie in all_movies]
    movie_texts = [movie_text(movie) for movie in all_movies]

    # TF-IDF i sličnost
    vectorizer = TfidfVectorizer(stop_words='english')
    tfidf_matrix = vectorizer.fit_transform(movie_texts)

    # Vektori filmova koje je korisnik ocijenio
    user_indices = [i for i, mid in enumerate(movie_ids) if mid in user_rated_ids]
    if not user_indices:
        return jsonify([])

    user_profile = np.asarray(np.mean(tfidf_matrix[user_indices], axis=0))

    similarities = cosine_similarity(user_profile, tfidf_matrix).flatten()

    # Rangiraj filmove po sličnosti (preskoči već gledane)
    recommended = []
    for idx in np.argsort(similarities)[::-1]:
        movie = all_movies[idx]
        if movie['id'] not in user_rated_ids:
            recommended.append(movie)
        if len(recommended) >= 12:
            break

    return jsonify(recommended)
```

Slika 4.18 Isječak koda koji prikazuje kako se opisi filmova pretvaraju u numeričke vektore TF-IDF metodom, izračunava se sličnost između korisničkog profila i svih filmova, te se na temelju toga generiraju preporuke temeljene na sadržaju.

Za naprednije preporuke koristi se i model strojnog učenja temeljen na algoritmu singularna dekompozicija vrijednosti (SVD) iz Python knjižnice *Surprise*. Ovaj model radi na čitavoj matrici korisničkih ocjena, a nakon treniranja može predvidjeti koliko bi neki korisnik mogao ocijeniti film koji još nije gledao. Singularna dekompozicija vrijednosti model trenira se nad skupom korisničkih ocjena, pri čemu se podaci uređuju u odgovarajući format za treniranje. U standardnoj implementaciji model koristi sve dostupne podatke, no za vrednovanje kvalitete preporuka podaci se mogu podijeliti na skup za treniranje i skup za testiranje. Vrednovanje se provodi mjerenjem pogreške predviđanja, primjerice korijen srednje kvadratne pogreške (RMSE) ili metrikom srednje apsolutne pogreška (MAE), čime se može objektivno procijeniti učinkovitost modela. U aplikaciji se model periodički trenira na najnovijim podacima, čime se osigurava svježina i relevantnost

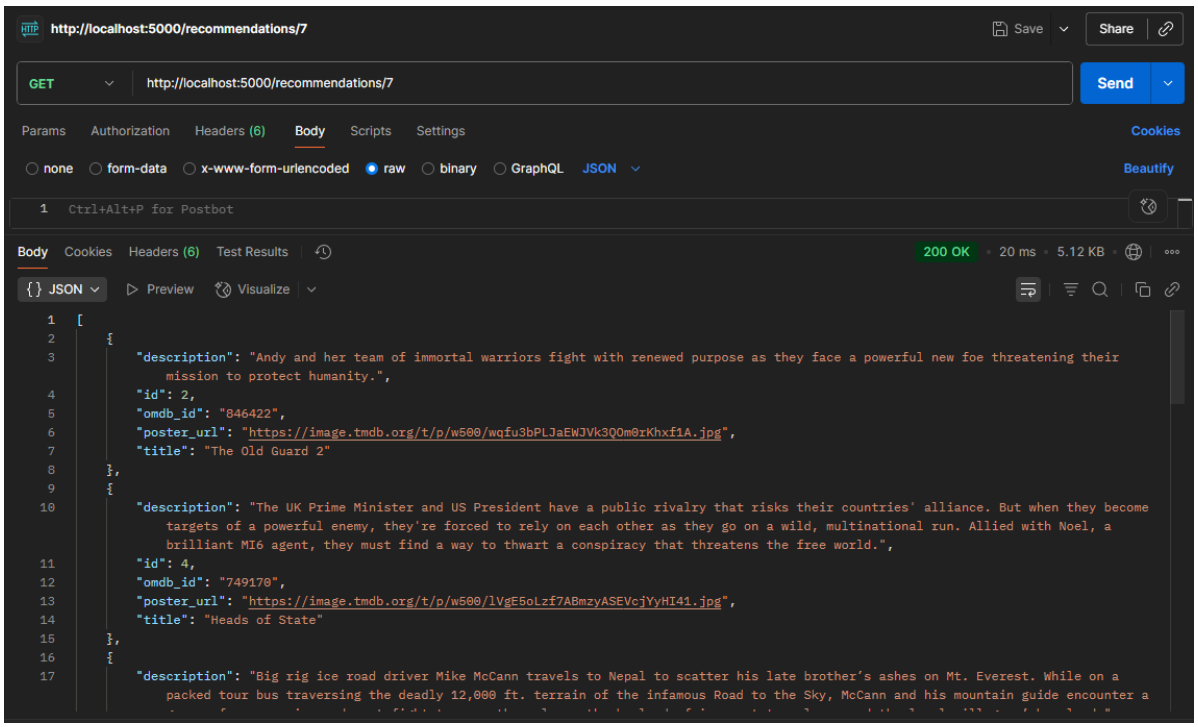
preporuka. Na slici 4.19 se prikazuje funkcija za treniranje i korištenje modela singularne dekompozicije vrijednosti. Prvo se dohvaćaju sve korisničke ocjene i podaci o filmovima, podatkovni skup se priprema za *Surprise* knjižnicu, a zatim se model trenira nad cijelim skupom podataka. Za svakog korisnika predviđaju se ocjene za filmove koje nije još ocijenio te se filmovi s najvišim predviđenim ocjenama predlažu kao preporuka. Kod uključuje i obradu grešaka te vraća rezultat u JSON formatu.

```python
@app.route('/ml_recommendations/<username>', methods=['GET'])
def ml_recommendations(username):
    try:
        from surprise import SVD, Dataset, Reader
        import numpy as np
        # 1. Dohvati sve ocjene
        all_ratings = get_all_ratings()
        if not all_ratings:
            return jsonify([])
        # 2. Dohvati sve filmove
        all_movies = get_all_movies_with_avg()
        if not all_movies:
            return jsonify([])
        # 3. Pripremi podatke za Surprise
        reader = Reader(rating_scale=(1, 5))
        data = Dataset.load_from_df(
            # Pretvori u DataFrame-like listu
            pd.DataFrame(all_ratings, columns=['user', 'item', 'rating']),
            reader
        )
        # 4. Treniraj model na svim podacima
        trainset = data.build_full_trainset()
        algo = SVD()
        algo.fit(trainset)
        # 5. Preporuči filmove koje korisnik nije ocijenio
        # Dohvati filmove koje je korisnik već ocijenio
        user_rated_ids = get_user_rated_movies(username)
        movie_ids = [movie['id'] for movie in all_movies]
        rec_candidates = [mid for mid in movie_ids if mid not in user_rated_ids]
        # Ako korisnik već sve ocijenio ili nema kandidata, vrati prazno
        if not rec_candidates:
            return jsonify([])
        # Predviđanje ocjene za svakog kandidata
        preds = []
        for mid in rec_candidates:
            try:
                pred = algo.predict(username, mid)
                preds.append((mid, pred.est))
            except Exception:
                continue
        # Sortiraj po predviđenoj ocjeni, vrati top 12
        preds.sort(key=lambda x: x[1], reverse=True)
        best_ids = [mid for mid, _ in preds[:12]]
        # Vrati podatke o filmovima
        recommended = [movie for movie in all_movies if movie['id'] in best_ids]
        return jsonify(recommended)
    except Exception as e:
        import traceback
        tb = traceback.format_exc()
        print(tb)
        return jsonify({"error": str(e), "traceback": tb}), 500
```

Slika 4.19 Prikaz funkcije za treniranje i korištenje singularne dekompozicije vrijednosti, modela strojnog učenja pri generiranju filmskih preporuka na temelju korisničkih ocjena.

4.5.3 Generiranje preporuka

Nakon što su svi modeli istrenirani i spremni za rad, aplikacija korisnicima omogućuje generiranje preporuka u stvarnom vremenu. Modeli za preporuke trenirani su na podacima o ocjenama i interakcijama korisnika iz interne *SQLite* baze, gdje svaka ocjena povezuje korisnika i film s vrijednostima od 1 do 5. Kod treniranja modela singularne dekompozicije vrijednosti, podatkovni skup se može podijeliti na skup za treniranje i skup za vrednovanje, čime se procjenjuje točnost modela pomoću metrika kao što su korijen srednje kvadratne pogreške i metoda srednje apsolutne pogreške. Na taj način se osigurava da modeli nisu pretrenirani te da daju relevantne preporuke u stvarnim uvjetima korištenja aplikacije. Svaki put kada korisnik putem sučelja odabere željeni tip preporuke, programsko rješenje na strani poslužitelja aplikacije obrađuje zahtjev primjenjujući odgovarajući model i generira novu listu preporučenih filmova. Ovisno o odabranom pristupu, preporuke se razlikuju: kod kolaborativnog filtriranja korisnik dobiva prijedloge na temelju ukusa sličnih korisnika, dok se kod pristupa temeljenog na sadržaju nude filmovi najbliži onima koje je korisnik već pozitivno ocijenio. S druge strane, model singularne dekompozicije vrijednosti, za svakog korisnika predviđa ocjene za sve filmove koje još nije gledao i preporučuje one s najvišim predviđenim ocjenama a na slici 4.20 prikazan je API zahtjev za preporuke gdje se vraća lista preporuka a na slici 4.21 prikazan je kod programskog rješenja na strani korisnika za prikaz preporuka ovisno o odabranom tipu preporuka.



Slika 4.20 Primjer API zahtjeva za preporuke i JSON odgovora s listom preporučenih filmova.

```
function prikaziPreporuke(tip) {
  ['collab','content','ml'].forEach(t => {
    const el = document.getElementById('tab-' + t);
    if (el) el.classList.remove('active');
  });
  const tabEl = document.getElementById('tab-' + tip);
  if (tabEl) tabEl.classList.add('active');

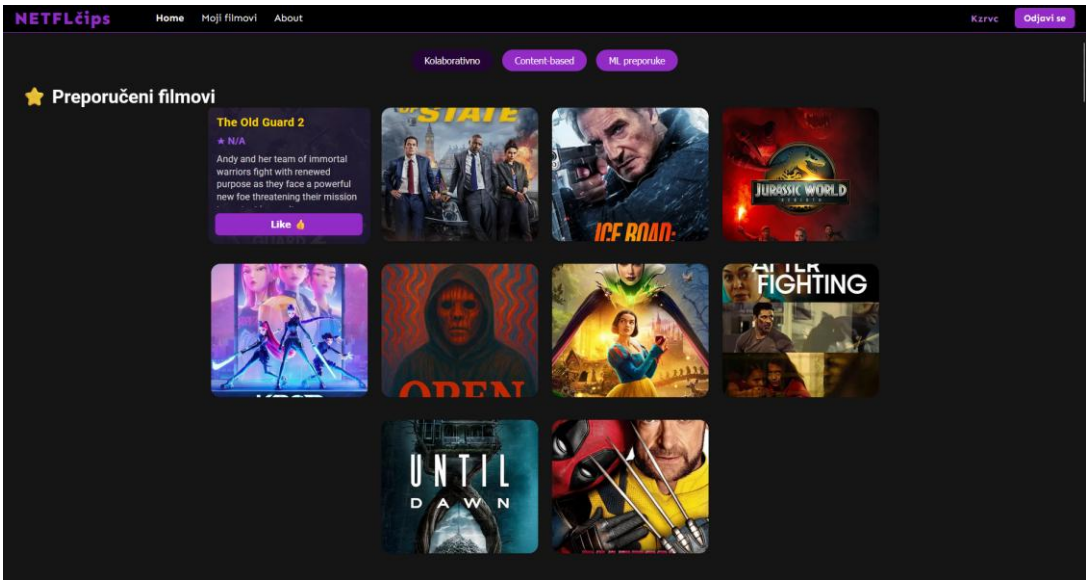
  const loader = document.getElementById('rec-loader');
  const listDiv = document.getElementById('recommended-list');
  loader.style.display = 'block';
  listDiv.innerHTML = '';

  const username = localStorage.getItem('username');
  let endpoint = '';
  if (tip === 'collab') endpoint = `/recommendations/${username}`;
  else if (tip === 'content') endpoint = `/content_recommendations/${username}`;
  else if (tip === 'ml') endpoint = `/ml_recommendations/${username}`;

  fetch(`http://127.0.0.1:5000${endpoint}`)
    .then(res => res.json())
    .then(films => {
      loader.style.display = 'none';
      listDiv.innerHTML = '';
      if (!films || films.length === 0) {
        listDiv.innerHTML = `<div class="empty-list-msg">Trenutno nema preporuka za ovaj način.</div>`;
        return;
      }
      films.forEach(film => {
        listDiv.innerHTML += `
          <div class="movie-list-item">
            <img class="movie-list-item-img" src="${film.poster_url || 'img/placeholder.jpg'}" alt="${film.title}">
            <div class="movie-info-overlay">
              <div class="movie-title">${film.title}</div>
              <div class="movie-rating">★ ${film.average_rating !== null
                && film.average_rating !== undefined ? film.average_rating.toFixed(2) : 'N/A'}</div>
              <div class="movie-desc">${film.description || 'Nema opisa.'}</div>
              <button class="movie-list-item-button" onclick="lajkajFilm('${film.id}')">Like 👍</button>
            </div>
          </div>
        `;
      });
    })
    .catch(error => {
      loader.style.display = 'none';
      listDiv.innerHTML = `<div class="empty-list-msg">Greška pri dohvaćanju preporuka.</div>`;
      console.error("Greška pri dohvaćanju preporuka s backend-a:", error);
    });
}
```

Slika 4.21 Funkcija programskog rješenja na strani korisnika za dohvat i prikaz preporučenih filmova korisniku ovisno o odabranom tipu preporuka.

Rezultati preporuka automatski se prikazuju korisniku na web sučelju, gdje su jasno istaknuti nazivi filmova, posteri, prosječne ocjene i kratki opisi kako je i prikazano na slici 4.22 Važna prednost ovakvog sustava je njegova dinamičnost: korisnik može u svakom trenutku promijeniti način dobivanja preporuka ili ocijeniti nove filmove, čime se preporuke odmah prilagođavaju njegovom najnovijem ukusu i aktivnostima. Takav pristup omogućuje personalizirano i korisnički orijentirano iskustvo pretraživanja filmskog sadržaja.



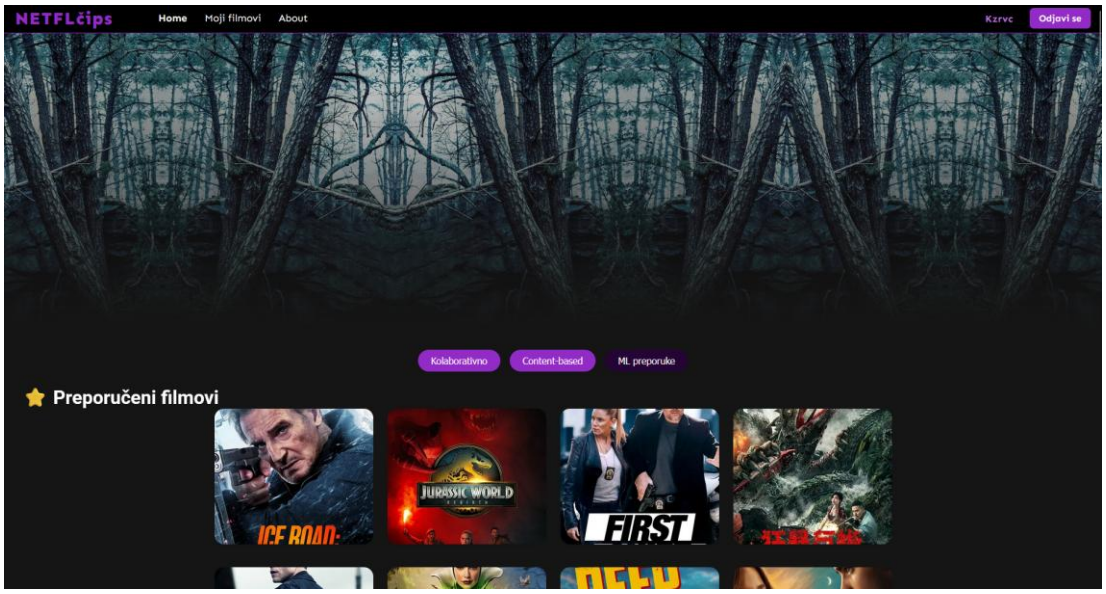
Slika 4.22 Prikaz preporučenih filmova korisniku u web aplikaciji, s vidljivim naslovima, posterima i ocjenama.

5 PRIKAZ RADA S ANALIZOM SUSTAVA I USPOREDBOM POSTUPAKA PREPORUČIVANJA

U ovom poglavlju prikazan je praktičan rad razvijenog web sustava za preporuku filmova, uz analizu učinkovitosti pojedinih metoda preporučivanja. Detaljno se prikazuje korisničko iskustvo pri korištenju aplikacije, uspoređeni su različiti pristupi filtriranju i analizira kvaliteta dobivenih preporuka. Također, istaknuta su ograničenja postojeće implementacije.

5.1 Prikaz korištenja web sustava

Korištenje web aplikacije za preporuku filmova dizajnirano je tako da bude pregledno i jednostavno za svakog korisnika, bez obzira o njegovom predznanju. Nakon uspješne prijave ili registracije, korisnik ima pristup glavnom sučelju gdje može pregledavati filmove, ocjenjivati ih, pretraživati bazu te generirati preporuke što je i prikazano na slici 5.1. Glavna značajka aplikacije je što omogućava odabir između tri različita modela preporuka: kolaborativnog filtriranja, filtriranja prema sadržaju i preporuka temeljenih na strojnome učenju (SVD).



Slika 5.1 Sučelje aplikacije s prikazom preporuka i opcijama za odabir modela preporučivanja.

Odabirom željenog modela preporuka putem označenih gumba ili kartica, korisniku se odmah prikazuje osvježena lista preporučenih filmova. Svaki film prikazuje poster, osnovne informacije, prosječnu ocjenu i kratak opis, što omogućuje brzo pregled i lak odabir. Korisnik može ocijeniti filmove jednostavnim klikom, a svaka ocjena automatski utječe na buduće preporuke. Zbog jednostavnog i prilagodljivog dizajna, sustav je pogodan i za korisnike sa manjim iskustvom.

Dodatno, sustav prikazuje osnovne statistike o broju ocijenjenih filmova, što korisnicima daje bolji pregled vlastitih filmskih sklonosti.

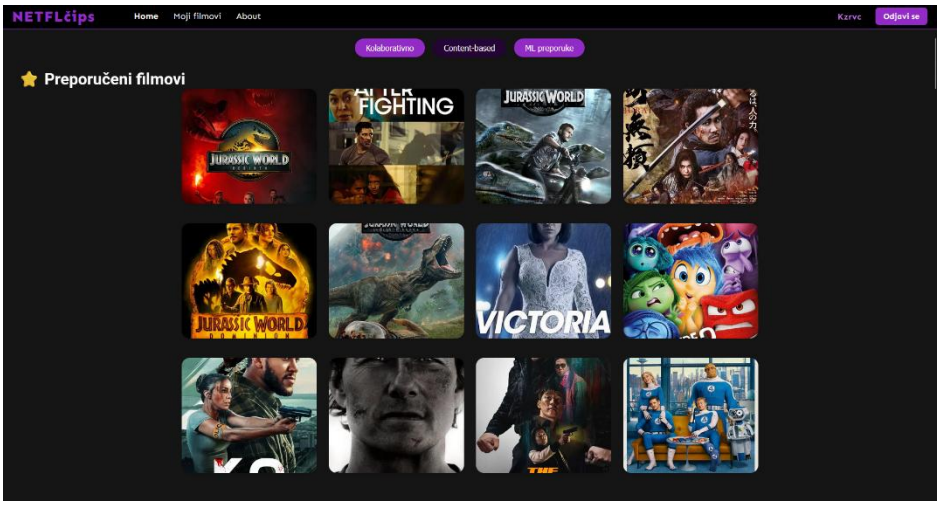
5.2 Usporedba filtriranja prema sadržaju, kolaborativnog filtriranja i modela strojnog učenja

U okviru razvijenog web sustava implementirane su tri različite metode preporučivanja filmova, što pruža mogućnost detaljne usporedbe njihovih prednosti i nedostataka na stvarnim primjerima. Filtriranje prema sadržaju temelji se na analizi osobina filmova, primjerice žanra, opisa i ključnih riječi, te korisnicima predlaže naslove koji su slični onima koje su ranije ocijenili. Ovaj pristup osigurava preporuke koje su konzistentne s ukusom korisnika, ali može ograničiti raznolikost jer često ostaje unutar poznatih okvira sklonosti.

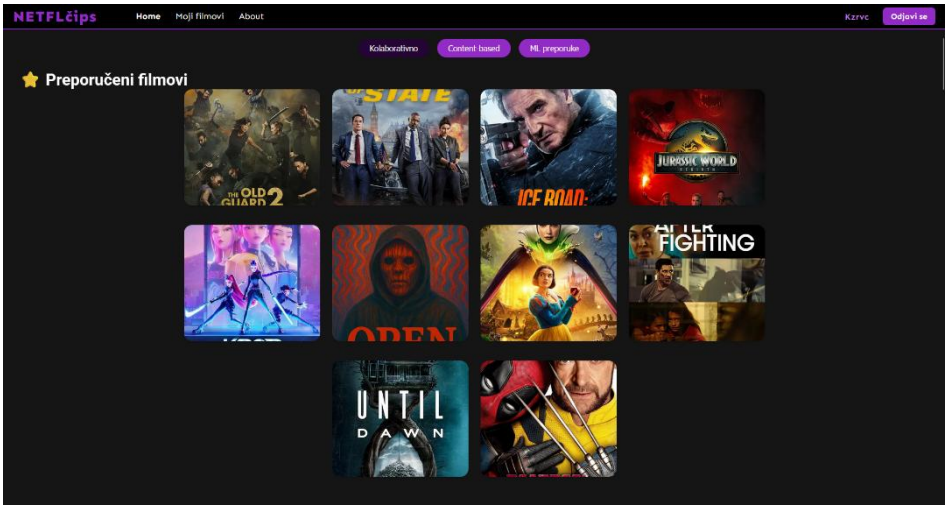
Kolaborativno filtriranje analizira obrasce ponašanja i ocjene većeg broja korisnika, pronalazi one sa sličnim ukusom te na temelju toga preporučuje filmove koje korisnik još nije pogledao. Prednost ovog pristupa je u tome što omogućuje otkrivanje novih i neočekivanih naslova, no njegova učinkovitost uveliko ovisi o veličini baze korisnika i količini prikupljenih ocjena.

Treći model koristi metode strojnog učenja, konkretno algoritam singularne dekompozicije vrijednosti. Ovaj pristup modelira cijelu matricu korisničkih ocjena i na temelju nje predviđa koliko bi se korisniku mogao svidjeti određeni film koji još nije gledao. Model strojnog učenja posebno se ističe kod korisnika s bogatom poviješću ocjena jer je tada sposoban prepoznati suptilnije obrasce u sklonosti.

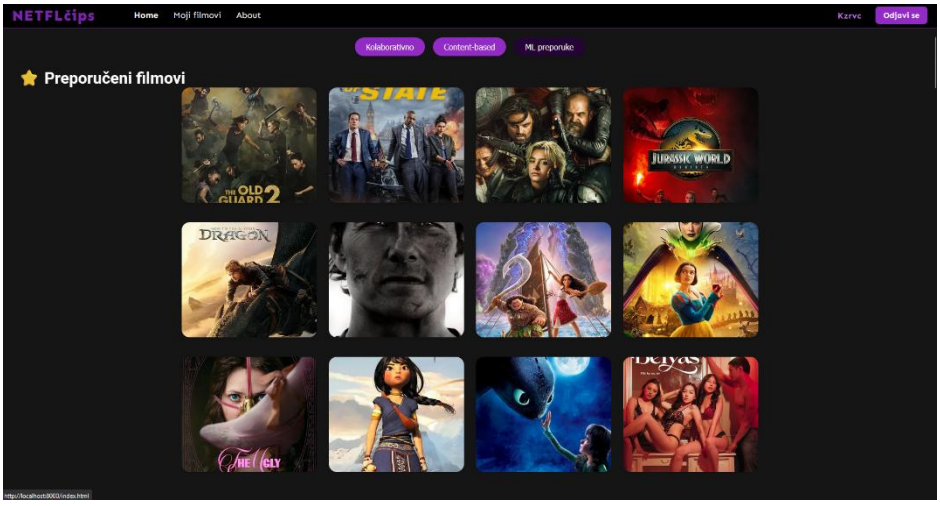
Usporedba rezultata pokazuje kako filtriranje prema sadržaju pruža najrelevantnije preporuke korisnicima s jasno izraženim žanrovskim sklonostima. Kolaborativno filtriranje i model strojnog učenja više predlažu raznovrsne filmove, uključujući i one izvan uobičajenih interesa korisnika. Model strojnog učenja često daje najpersonaliziranije i najpreciznije preporuke za aktivnije korisnike, dok je filtriranje prema sadržaju pogodnije za novije ili manje aktivne korisnike. Na slici 5.2, 5.3 i 5.4 prikazane su usporedbe preporuka u ovisnosti odabranog tipa preporuke.



Slika 5.2 Usporedba preporuka dobivenih kolaborativnim pristupom za istog korisnika.



Slika 5.3 Usporedba preporuka dobivenih filtriranjem prema sadržaju za istog korisnika.



Slika 5.4 Usporedba preporuka dobivenih pristupom strojnog učenja za istog korisnika.

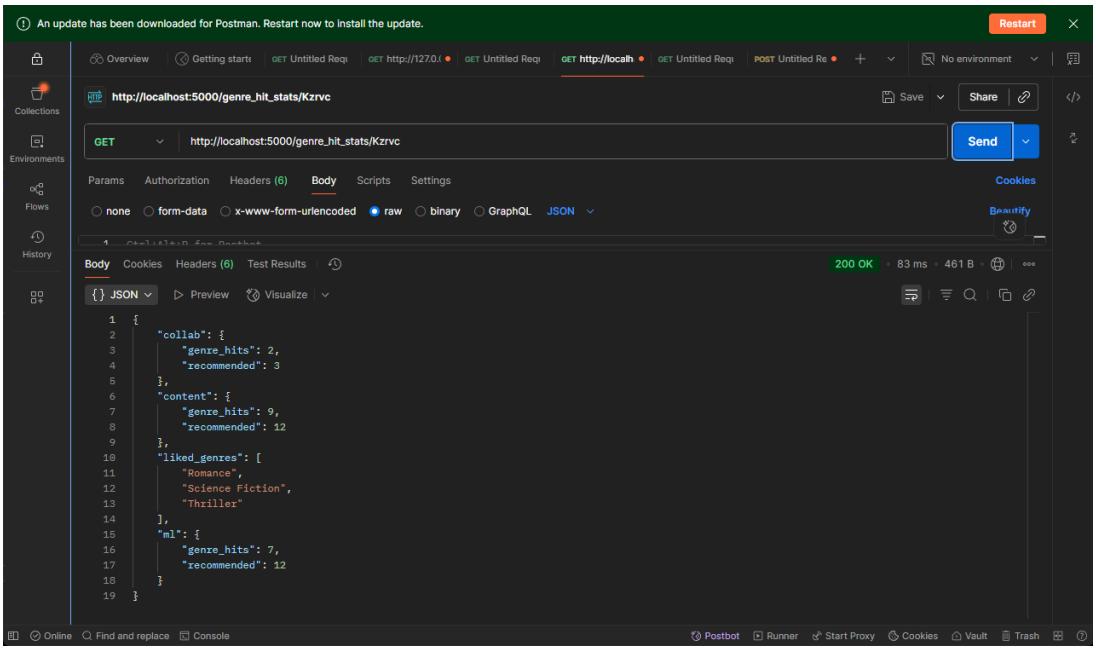
5.3 Analiza kvalitete preporuka

Kvaliteta preporuka procjenjivana je kroz nekoliko kriterija: relevantnosti, raznolikosti i zadovoljstvo korisnika. Osim standardne analize interakcija korisnika i povratnih informacija, dodatno je provedeno testiranje relevantnosti preporuka prema žanrovima filmova koje je korisnik prethodno označio kao omiljene.

Aplikacija sadrži funkcionalnost koja analizira žanrove preporučenih filmova i utvrđuje koliko njih odgovara korisnikovim preferencijama. Rezultati testiranja govore kako filtriranje prema sadržaju pokazuje najveću preciznost, od dvanaest preporučenih filmova, čak devet odgovara korisnikovim omiljenim žanrovima *(Romance, Science Fiction, Thriller*). Model strojnog učenja također pokazuje visoku relevantnost prema žanru (sedam od dvanaest preporuka), dok kolaborativno filtriranje ima nešto manju usklađenost (dva od tri preporučena filma).

Kvantitativna analiza uključuje praćenje prosječnih ocjena koje su korisnici dali preporučenim filmovima, broj lajkova i broj filmova koje su korisnici označili nakon preporuke. Kvalitativna analiza provedena se na anketi među korisnicima i povratnim informacijama prikupljenima putem aplikacije. Većina korisnika smatra kako su preporuke relevantne i korisne, ali primjećuju kako bi sustav mogao bolje izbjegavati ponavljanje već viđenih filmova. Ističe se kako pristup pomoću strojnog učenja najbolje personalizira preporuke za korisnike s velikim brojem ocjena, dok je filtriranje prema sadržaju pristup korisniji za početnike.

Provedenom analizom žanrovske relevantnosti potvrđuje se kako je sustav sposoban prepoznati i predlagati filmove koji odgovaraju stvarnim sklonostima korisnika, čime se dodatno potvrđuje kvaliteta i korisnost sustava stvaranja preporuka i slika 5.5 pokazuje rezultat povratne informacije.



Slika 5.5 Prikaz žanrovske statistike preporuka za različite modele. Model filtriranja prema sadržaju postiže najviše žanrovskih pogodaka, dok model strojnog učenja i kolaborativni pristupi također daju relevantne preporuke prema korisnikovim preferiranim žanrovima.

5.4 Ograničenja implementacije

Bez obzira o brojnim funkcionalnostima i prednostima koje sustav za preporuke filmova pruža, važno je istaknuti određena ograničenja koja utječu na njegovu ukupnu učinkovitost i korisničko iskustvo. Jedno važno ograničenje odnosi se na količinu i raznolikost dostupnih podataka. U ranim fazama korištenja, kada broj korisnika i ocjena još nije dovoljan, sustav se suočava s izazovima poznatim kao *cold start* preporuke često nisu dovoljno precizne jer nema dovoljno informacija za pouzdano prepoznavanje sklonostima korisnika. S povećanjem broja korisnika i interakcija, kvaliteta preporuka se postupno poboljšava.

Filtriranje prema sadržaju također ima svoja ograničenja, budući da njegova učinkovitost ovisi o kvaliteti i detaljnosti podataka o filmova. Ukoliko opisi filmovima nisu dovoljno informativni ili su žanrovi loše definirani, preporuke mogu biti manje relevantne i raznolike. Dok model temeljen na strojnom učenju zahtijeva stalno prikupljanje novih podataka te redovito treniranje kako bi ostao ažuran i prilagođen korisničkim navikama. Kod velikih baza podataka, to može predstavljati tehnički i vremenski izazov.

Trenutna inačica aplikacije nema napredne mogućnosti poput hibridnih modela, koji kombiniraju prednosti različitih algoritama, niti integriranje s vanjskim filmskim uslugama poput *IMDb*-a, što

bi obogatilo bazu podataka i dalo širu personalizaciju preporuka. Društvene preporuke i korištenje podataka iz korisničkih mreža nisu implementirani, iako su dokazano korisni u sličnim sustavima za preporuku.

Aplikacija je optimirana za korištenje u web okruženju, dok podrška za mobilne uređaje i dodatne opcije personalizacije još nisu razvijene u potpunosti. Buduće nadogradnje mogu uključivati dodatne funkcionalnosti, integraciju s vanjskim izvorima podataka te razvoj mobilne verzije, čime bi se sustav prilagodio većoj količini korisnika i poboljšao iskustvo korištenja sustava preporuka.

6 ZAKLJUČAK

U ovom završnom radu razvijen je web sustav za preporuku filmskih sadržaja koji korisnicima omogućuje jednostavno i personalizirano otkrivanje novih naslova filmova prema osobnim sklonostima. Implementacijom triju metoda preporuka i to filtriranja prema sadržaju, kolaborativnog filtriranja i modela strojnog učenja – sustav pruža relevantne, popularne i raznovrsne prijedloge, čime se podiže kvaliteta korisničkog iskustva na digitalnim platformama. Analiza rezultata pokazala je kako svaka od metoda za preporuku koje su implementirane imaju svoje prednosti, tako pristup zasnovan na sadržaju omogućuje visoku relevantnost kod korisnika sa specifičnim zanimanjima, dok s druge strane kolaborativno filtriranje i strojno učenje omogućuju otkrivanje novih filmova i veću raznolikost preporuka. Tijekom testiranja aplikacije prikazana je njena funkcionalnost i prilagodljivost ovisno o različitim profilima korisnika. Iako sustav ostvaruje visoku razinu personalizacije, zapažena su ograničenja, kao što su problemi hladnog starta kod novih korisnika i potreba za kvalitetnim opisima filmova. Sustav se može unaprijediti implementacijom hibridnih modela za preporuku, integracijom s vanjskim bazama podataka te proširenjem funkcionalnosti za mobilne uređaje. Web aplikacija razvijena u ovom radu je vrijedan i važan alat za poboljšanje iskustva korisnika pri pretraživanju filmskih sadržaja. Nadogradnja aplikacije može još podići kvalitetu preporuka i omogućiti širu personalizaciju sadržaja, što korisnicima pruža podršku u otkrivanju i praćenju njihovih filmskih sadržaja.

7 LITERATURA

[1] J. Bobadilla, F. Ortega i A. Hernando, Recommender systems servev, Knowledge-Based Systems, 2013.

[2] Mann i M. Sharma, »A Survev of Recommender Systems: Approaches and Limitations,« 2013.

[3] L. Plummer, » This Is How Netflix's Top-Secret Recommendation System Works,« Wired, 2017. [Mrežno]. Available: https://www.wired.com/story/how-do-netflixs-algorithms-work-machine-learning-helps-to-predict-what-viewers-will-like/. [Pokušaj pristupa 28 05 2025].

[4] »What is Netflix,« Netflix, 2018. [Mrežno]. Available: https://help.netflix.com/en/node/412. [Pokušaj pristupa 27 07 2025].

[5] »What is Youtube,« Webwise, 15 Srpanj 2023. [Mrežno]. Available: https://www.webwise.ie/parents/what-is-youtube/. [Pokušaj pristupa 27 07 2025].

[6] IMDb, »About IMDb,« 2024. [Mrežno]. Available: https://www.imdb.com/pressroom/about/. [Pokušaj pristupa 27 7 2025].

[7] R. Burke, A. Felfernig i H. Göker, »Recommender Systems: An Overview,« *AI Magazine,* 2011.

[8] F. Ricci, L. Rokach i B. Shapira, Recommender Systems Handbook, New York, 2015.

[9] M. Foundation, »Learn Web Development,« Mozilla Foundation, 2024. [Mrežno]. Available: https://developer.mozilla.org/en-US/docs/Learn. [Pokušaj pristupa 25 05 2025].

[10] F. Rozanski, »The key to system quality: Non-functional requirements,« Managed services, 2024. [Mrežno]. Available: https://spyro-soft.com/blog/managed-services/the-key-to-system-quality-non-functional-requirements. [Pokušaj pristupa 23 06 2025].

[11] G. Developers, »Content-Based Filtering Advantages & Disadvantages,« Google for Developers. [Mrežno]. Available: https://developers.google.com/machine-learning/recommendation/content-based/summary. [Pokušaj pristupa 20 06 2025].

[12] T. Staff, »A Guide to Content-Based Filtering in Recommender Systems,« 2022. [Mrežno]. Available: https://www.turing.com/kb/content-based-filtering-in-recommender-systems. [Pokušaj pristupa 21 06 2025].

[13] S. Lewandowski, »Content-Based Filtering Pros and Cons,« 2023. [Mrežno]. Available: https://www.szymonlewandowski.pl/blog/recommender-systems-content-based-filtering. [Pokušaj pristupa 22 06 2025].

[14] IBM, »What is Collaborative Filtering?,« IBM, 2023. [Mrežno]. Available: https://www.ibm.com/topics/collaborative-filtering. [Pokušaj pristupa 28 05 2025].

[15] D. Pandithu, »Recommender system user collaborative filtering,« 09 2024. [Mrežno]. Available: https://medium.com/@deepapandithu/recommender-system-user-collaborative-filtering-37613f0c6a9. [Pokušaj pristupa 28 04 2025].

[16] B. Sarwar, »Item-Based Collaborative Filtering Recommendation Algorithms,« 2001. [Mrežno]. Available: https://www.researchgate.net/publication/2369002_Item-based_Collaborative_Filtering_Recommendation_Algorithms. [Pokušaj pristupa 24 07 2025].

[17] Alshbanat, Benhidour i Kerrache., »A Survey of Latent Factor Models in Recommender Systems,« 2024. [Mrežno]. Available: https://arxiv.org/abs/2405.18068.

[18] S. Airen, »TopK Movie Recommendation Using Matrix Factorization Methods,« u *Springer*, https://link.springer.com/chapter/10.1007/978-981-97-4860-0_14, 2023.

[19] AutoLFA, »A Hybrid Recommender System Based on Autoencoder and Latent Feature Analysis,« *MDPI,* pp. https://www.mdpi.com/1099-4300/25/7/1062, 2023.

[20] Pandey i Ayush, »Personalized Film Recommendations Using Machine Learning,« 2025. [Mrežno]. Available: https://www.iosrjournals.org/iosr-jce/papers/Vol27-issue2/Ser-2/F2702023137.pdf.

[21] Foundation i Python, »What is Python? Executive Summary,« Python, 2024. [Mrežno]. Available: https://www.python.org/doc/essays/blurb/. [Pokušaj pristupa 25 05 2025].

[22] Pallets, »Flask Documentation,« Flask, 2024. [Mrežno]. Available: https://flask.palletsprojects.com/en/2.3.x/. [Pokušaj pristupa 25 05 2025].

[23] W3Schools, »AJAX Introduction,« W3Schools, 2024. [Mrežno]. Available: https://www.w3schools.com/js/js_ajax_intro.asp. [Pokušaj pristupa 25 05 2025].

SAŽETAK

U ovom završnom radu razvijena je web aplikacija za preporuku filmskih sadržaja s ciljem poboljšanja korisničkog iskustva pri pretraživanju i odabiru filmova na digitalnim platformama. Aplikacija je implementirana korištenjem programskog jezika Python i web okvira *Flask*, dok je korisničko sučelje izrađeno pomoću tehnologija HTML-a, CSS-a, *JavaScripta* i *Bootstrapa*. Sustav omogućuje registriranje korisnika, ocjenjivanje filmova, pregled preporuka i upravljanje osobnim sklonostima. Rad je usmjeren na primjenu i usporedbu tri pristupa generiranju preporuka: filtriranje prema sadržaju, kolaborativno filtriranje te metodu strojnog učenja, odnosno algoritam singularne dekompozicije vrijednosti. Provedeno je vrednovanje preporuka na testnom skupu podataka, a rezultati pokazuju visoku razinu relevantnosti i personalizacije za različite tipove korisnika. U radu su također identificirana ograničenja razvojnog rješenja, poput problema hladnog starta i potrebe za kvalitetnim podatcima. U radu se pokazuje mogućnosti primjene naprednih algoritama preporuke u stvarnom web okruženju i doprinosi razvoju učinkovitijih digitalnih alata za odabir filmskog sadržaja.

Ključne riječi: filtriranje prema sadržaju, filmska preporuka, kolaborativno filtriranje, strojno učenje, web aplikacija

ABSTRACT

In this final thesis, a web application for movie content recommendation was developed with the aim of improving the user experience in searching for and selecting films on digital platforms. The application was implemented using the Python programming language and the Flask web framework, while the user interface was created with HTML, CSS, JavaScript, and Bootstrap technologies. The system allows user registration, movie rating, recommendation browsing, and management of personal preferences. The work focuses on the application and comparison of three approaches to generating recommendations: content-based filtering, collaborative filtering, and a machine learning method, namely the singular value decomposition algorithm. Recommendation evaluation was carried out on a test dataset, and the results demonstrate a high level of relevance and personalization for different types of users. The thesis also identifies limitations of the development solution, such as the cold start problem and the need for high-quality data. This work demonstrates the potential of applying advanced recommendation algorithms in a real web environment and contributes to the development of more efficient digital tools for movie content selection.

Keywords: content-based filtering, film recommendation, collaborative filtering, machine learning, web application

ŽIVOTOPIS

Anto Kožarević rođen je 30. siječnja 2003. godine u Slavonskom Brodu. Nakon pohađanja Osnovne škole Vladimira Nazora u Odžaku, upisuje Srednju školu Pere Zečevića u Odžaku smjer tehničar za računarstvo. Poslije srednje škole upisuje Fakultet elektrotehnike, računarstva i informacijskih tehnologija Osijek, preddiplomski sveučilišni studij Računarstva, smjer Računalno inženjerstvo.

PRILOZI

Prilog 1. Završni rad u datoteci docx.

Prilog 2. Završni rad u datoteci pdf.

Prilog 3. Git repozitorij s programskim kodom aplikacije